

LLM FINE- TUNING USING PERFORMANCE EFFICIENT FINE- TUNING (PEFT)
METHODS

Swapnil Anil Janorkar

Final Thesis Report for
Master of Science in Machine Learning & Artificial Intelligence
Liverpool John Moores University & upGrad

May 2024

ACKNOWLEDGEMENT

I would like to sincerely thank my research supervisor Dr. Harika Vuppala for her valuable advice, suggestions, guidance, and encouragement throughout the entire duration of my master's thesis.

I would deeply like to extend my gratitude to my friends, family and colleagues for supporting and encouraging me since the beginning of this program. Special shout out to my wife Suvarna Ashok Kshirsagar for patiently listening to all my queries, doubts, providing her valuable advice and help in proofreading.

Finally, I would like to thank Upgrad and LJMU for providing an opportunity and all the necessary resources required for the master's thesis.

ABSTRACT

Since the launch of ChatGPT by OpenAI in November 2022, a new boom in Artificial Intelligence has been started, especially in the field of Large Language Models and Foundation Models. Almost all the Natural Language Processing (NLP) tasks like Sentiment Analysis, Topic Modelling, Machine Translation, Named Entity Recognition (NER), etc. can be accomplished by modern LLMs. These LLMs are trained with a vast amount of generic data available freely on the internet using specialized hardware called Graphics Processing Unit (GPU). Training the LLM is a cost and time-intensive task. According to the online resources, the OpenAI GPT-3 model was trained using 1024 NVIDIA V100 GPUs for 34 days and cost around USD 4.6M. These general-purpose LLMs may not perform well for real-life business use cases. To cater to such business use cases LLMs need to be fine-tuned with private or enterprise data. In real-life all the data might not be available at once for the LLM fine-tuning process. Data might be available periodically or at regular intervals. This research tries to find out whether LLMs should be fine-tuned more frequently with smaller datasets or less frequency with larger datasets. This research will help AI Engineers and Data Scientists to decide the frequency of model fine-tuning and the amount of data to be used. LLM inference accuracy, resources required, and time taken for fine-tuning process will be considered for determining the overall performance of both approaches.

LIST OF TABLES

Table 1: LoRA fine-tuning resource utilization in percentage61

Table 2: QLoRA fine-tuning resource utilization in percentage62

Table 3: LoRA fine-tuning time taken64

Table 4: QLoRA fine-tuning time taken.....65

Table 5: LoRA fine-tuning Inference Accuracy.....68

Table 6: QLoRA fine-tuning Inference accuracy69

LIST OF FIGURES

Figure 1: LLM Pre-Training and Fine-tuning Process	33
Figure 2: Parameter-Efficient Fine-Tuning using LoRA	35
Figure 3: E2E LLM Fine-Tuning Pipeline	39
Figure 4: Sample instruction from the dataset.....	44
Figure 5: Sample output from the dataset.....	44
Figure 6: Histogram displaying Instruction length.....	45
Figure 7: Box plot of the Instruction length	45
Figure 8: Histogram displaying Output length.....	46
Figure 9: Box plot of the Output length	46
Figure 10: Instruction Fine-Tuning prompt format	47
Figure 11: Full Data Fine-tuning Flow Chart.....	51
Figure 12: Code snippet to download model & tokenizer from huggingface	51
Figure 13: Llama-3 Inference code snippet.....	53
Figure 14: Split Data Fine-tuning Flow Chart.....	55
Figure 15: Code snippet to load model from local disk	56
Figure 16: Zero-Shot prompt example	57
Figure 17: nvidia-smi command to collect the GPU metric every 10 seconds	58
Figure 18: LoRA fine-tuning resource requirement.....	61
Figure 19: QLoRA fine-tuning resource requirement	62
Figure 20: LoRA fine-tuning time taken	64
Figure 21: QLoRA fine-tuning time taken	65
Figure 22: Prompt to generate Python code with expected time and space complexity	66
Figure 23: Prompt to generate Python code without special instructions	66
Figure 24: GPT-2 Full Dataset fine-tuning inference with generated Python code	67
Figure 25: GPT-2 Full Dataset fine-tuning inference without Python code.....	67
Figure 26: Fine-tuned LoRA model generated Python Code	68
Figure 27: Fine-tuned QLoRA model generated Python Code	69

LIST OF ABBREVIATIONS

AI: Artificial Intelligence

AWS: Amazon Web Services

CNN: Convolutional Neural Network

CPU: Central Processing Unit

EC2: Elastic Cloud Compute

FDFT: Full Dataset Fine-Tuning

GB: Giga Bytes

GEN AI: Generative Artificial Intelligence

GPT: Generative Pre-Trained Transformers

GPU: Graphics Processing Unit

GRU: Gated Recurrent Unit

LLM: Large Language Models

LoRA: Low Rank Adaptation

LSTM: Long Short Term Memory

ML: Machine Learning

NLP: Natural Language Processing

QLoRA: Quantized Low Rank Adaption

PEFT: Parameter Efficient Fine-Tuning

PTL: Pre Trained Language

ReLU: Rectified Linear Unit

RNN: Recurrent Neural Network

SDFT: Split Dataset Fine-Tuning

SLM: Small Language Models

TIIUAE: Technology Innovation Institute United Arab Emirates

USA: United States of America

V&L: Vision & Language

Table of Contents

ACKNOWLEDGEMENT	2
CHAPTER 1: INTRODUCTION	11
1.1 Background.....	11
1.2 Problem Statement	12
1.3 Aim and Objectives.....	13
1.4 Scope of the Study.....	14
1.5 Significance of the Study	15
1.6 Structure of the Study.....	16
CHAPTER 2: LITERATURE REVIEW.....	17
2.1 Introduction	17
2.2 Transformer Architecture.....	17
2.1.1 Introduction.....	17
2.1.2 Attention Mechanism	17
2.1.3 Transformer Model	18
2.1.3.1 Encoder and Decoder.....	18
2.1.3.2 Attention	18
2.1.3.2.1 Scaled Dot-Product Attention	19
2.1.3.2.2 Multi-Head Attention.....	19
2.1.3.2.3 Position-wise Feed-Forward Networks	19
2.1.3.2.4 Embeddings and Softmax	20
2.3 Pre-Trained Language (PTL) Model Fine-Tuning.....	20
2.3.1 Introduction	20
2.3.2 Parameter Efficient Fine-Tuning (PEFT)	20
2.3.2.1 Low Rank Adaptation (LoRA)	21
2.3.2.2 Quantized Low-Rank Adaptation (QLoRA).....	22
2.3.2.3 Prefix Tuning.....	23
2.3.2.4 Prompt Tuning	24
2.4 Distributed Parameter-Efficient Fine-Tuning (DLoRA)	25
2.5 Multiple-Trained LoRA Composition	26
2.6 LLM Fine-Tuning and Hallucination.....	27
2.6.1 Types of LLM Hallucination	27
2.6.1.1 Factuality Hallucination.....	27
2.6.1.2 Faithfulness Hallucination	28
2.6.2 Causes of Hallucination.....	28

2.6.2.1 Hallucination from Data	28
2.6.2.2 Hallucination from Training	28
2.6.2.3 Hallucination from Inference.....	29
2.6.3 Model Fine-Tuning and Hallucination.....	29
2.7 Hallucination Evaluation	29
2.8 Summary	31
CHAPTER 3: RESEARCH METHODOLOGY	32
3.1 Introduction	32
3.2 Large Language Models & PEFT Techniques	32
3.2.1 Large Language Model (LLM).....	32
3.2.2 LLM Fine-Tuning.....	32
3.2.3 Types of LLM Fine-Tuning	33
3.2.3.1 Full Parameter Fine-Tuning.....	34
3.2.3.2 Parameter-Efficient Fine-Tuning	34
3.2.3.2.1 Low Rank Adaptation (LoRA)	35
3.2.3.2.2 Quantized Low-Rank Adaptation (QLoRA)	35
3.3 Transformer Based Language Models	36
3.3.1 Falcon-RW-1B	36
3.3.2 Phi-1	37
3.3.3 Phi-1.5.....	37
3.3.4 TinyLlama_v1.1	37
3.3.5 GPT-2	38
3.4 Methodology	39
3.4.1 End-to-End Pipeline	39
3.4.1.1 Load Dataset, Tokenizer and Model	39
3.4.1.2 Set PEFT & Training Parameters	40
3.4.1.3 Generate Test Prompts	40
3.4.1.4 Pre-Fine-Tuning Inference	40
3.4.1.5 LLM Fine-Tuning.....	40
3.4.1.6 Post-Fine-Tuning Inference	40
3.4.1.7 Evaluation	40
3.4.2 Datasets	41
3.4.2.1 microsoft/orca-math-word-problems-200k.....	41
3.4.2.2 vezora/Tested-22k-Python-Alpaca	41
3.5 Tools	42

3.5.1 Hardware	42
3.5.2 Software	42
3.6 Summary	43
CHAPTER 4: IMPLEMENTATION DETAILS	44
4.1 Introduction	44
4.2 Dataset Details	44
4.3 Exploratory Data Analysis	45
4.4 Data Preparation	47
4.5 Hyperparameters	48
4.5.1 Training Parameters:	48
4.5.2 LoRA Configuration:	49
4.6 LLM Fine-Tuning	50
4.6.1 Full Dataset Fine-Tuning (FDFT):	50
4.6.2 Split Dataset Fine-Tuning (SDFT):	54
4.7 Model Inference	57
4.8 GPU & Memory Metric Collection	58
4.9 AWS SageMaker	58
4.10 Summary	59
CHAPTER 5: RESULTS AND EVALUATIONS.....	60
5.1 Introduction	60
5.2 Resource Requirements	60
5.3 Time Required for Fine-Tuning.....	63
5.4 Accuracy of the LLM Inference	66
5.5 Summary	70
CHAPTER 6: CONCLUSION AND RECOMMENDATIONS	71
6.1 Introduction	71
6.2 Conclusions.....	71
6.2.1 Model Fine-Tuning Resource Consumption.....	71
6.2.2 Faster Model Fine-Tuning.....	71
6.2.3 Inference Accuracy	72
6.3 Future Work and Recommendations	72
6.3.1 Use Better GPU	72
6.3.2 Use of Larger Datasets	72
6.3.3 Other Parameter-Efficient Fine-Tuning Methods.....	73

6.3.4 Hyperparameter Tuning.....	73
6.4 Contribution	73
6.5 Summary	73
APPENDIX A: RESEARCH PROPOSAL	79
APPENDIX B: LLM FINE-TUNING IMPLEMENTATION IN PYTHON	94

CHAPTER 1: INTRODUCTION

1.1 Background

In 2017, Google published a paper called “**Attention is All You Need**” (Vaswani et al., 2017) and introduced a new Neural Network Architecture called Transformer. The introduction of transformer architecture has laid the groundwork for advanced Machine Learning (ML) and Natural Language Processing (NLP) models. Transformer uses a concept called Attention Mechanism, which allows the neural network to focus on a specific part of the input. It also processes the input data sequentially, making it faster than the Recurrent Neural Network (RNN) and Convolutional Neural Network (CNN) models. All the modern Large Language Models (LLM) are based on the transformer architecture introduced by Google.

OpenAI introduced ChatGPT (Generative Pre-Trained Transformers) in October 2022 and since then a new era in the Foundation Model and Large Language Models (LLM) domain has started. All the major technology companies have already developed various general-purpose LLMs; like GPT-2, GPT-3, GPT-3.5, GPT-4o from OpenAI (Brown et al., 2020), Llama2, Llama3 and Code Llama from Meta (Touvron et al., 2023), Gemini from Google, Claude from Anthropic, Mistral 7b, Mixtral 8x7b and Mixtral 8x22b from Mistral AI, Falcon from tiuuai (Almazrouei et al., 2023) and Titan from Amazon Web Services (AWS).

These LLMs are pre-trained on large volumes of public data freely available over the internet. These general-purpose foundation models are very good at performing a wide range of natural language processing (NLP) tasks. e.g. Question-Answer, Text Generation, Summarization, Translation, Text Completion, Semantic Search, etc.

However, these LLMs perform very poorly and are inefficient when it comes to domain-specific questions. Fields like Medical, Legal, Science, Sports etc. As these models are not pre-trained with any domain-specific data.

These LLMs can be re-trained to perform some domain-specific tasks more efficiently and accurately using the process called LLM fine-tuning. GitHub Co-Pilot and Meta Code Llama are fine-tuned versions of GPT-3 (Brown et al., 2020) and Llama2 (Touvron et al., 2023) for

writing programming code and providing coding suggestions to the Software Engineers and Developers.

So, in short, a general-purpose language model can be turned into a specialist in a certain field/domain by fine-tuning it with domain-specific data. LLM fine-tuning is a data, resource, and time-intensive process. It requires an enormous volume of specialized labeled datasets along with a large computing power of expensive GPUs. It takes several days or weeks to complete the model fine-tuning.

LLM has a wide variety of applications in the field of Natural Language Processing. Almost all the NLP tasks can be done using the LLMs, but in some specific business cases, LLMs need to be fine-tuned to achieve the desired results. e.g. Sentiment analysis, Named Entity Recognition, Text Summarization, and Topic Modelling for custom datasets.

GitHub Co-Pilot which is based on OpenAI's GPT-3 and GPT-4, is fine-tuned using various code samples available on GitHub repositories and other public code repositories to generate the coding suggestions.

The purpose of this research is to find out the effect of the fine-tuning frequency and volume of datasets used on the LLM performance, resources required, and total time taken for fine-tuning the LLM. This study also aims to help Data Scientists and AI Engineers decide the frequency of LLM fine-tuning and the amount of data to be used for optimal use of resources without compromising the accuracy and performance of LLM.

1.2 Problem Statement

Study the effects of frequent fine-tuning using the Parameter-Efficient Fine-Tuning (Ding et al., 2023) methods at shorter time intervals with smaller datasets versus less frequent fine-tuning using the Parameter-Efficient Fine-Tuning (Ding et al., 2023) at longer intervals with large datasets on LLM accuracy, performance, time required, and resource utilization.

Study aims to help Data Scientists and AI Engineers to strike out the balance between dataset volume and training interval for optimal use of resources and to achieve the highest quality of model inference.

1.3 Aim and Objectives

In real-life cases, a large volume of training datasets might not be readily available for fine-tuning the LLM for some specific business use case. This research will help to decide the frequency of fine-tuning the LLM with domain-specific datasets and the volume of datasets to be used.

This research paper tries to find out model performance, model accuracy, and resource consumption when LLM is fine-tuned more frequently with smaller datasets versus less frequently with larger datasets.

Objectives:

1. To explore the Parameter-Efficient Fine-Tuning (PEFT) (Ding et al., 2023) methods LoRA (Hu et al., n.d.) and QLoRA (Dettmers et al., 2023).
2. To explore the different open-source Small Language Models to be used for fine-tuning.
 1. tiieau/Falcon-RW-1B (Almazrouei et al., 2023).
 2. TinyLlama/TinyLlama_v1.1
 3. Openai-community/GPT-2 (Brown et al., 2020)
 4. Microsoft/phi-1& phi-1.5
3. To explore the HuggingFace platform and Python open-source libraries published by HuggingFace to download the model and datasets and implement the model fine-tuning.
 1. transformers v4.43.3
 2. datasets v2.19.2
 3. peft v0.12.0
 4. bitsandbytes v0.41.3
 5. trl v0.8.6

4. To fine-tune the model using the PEFT (Ding et al., 2023) method with all the training datasets at once versus splitting the large dataset into multiple smaller datasets of equal size. Analyse both the approaches against following metric.
 1. CPU/GPU requirement.
 2. Memory requirement.
 3. Time taken to complete the fine-tuning process.
 4. Model Inference accuracy and model performance.
5. To understand and use the AWS SageMaker Notebooks to run the model fine-tuning process.
6. To utilize the NVIDIA A10G GPU for the model fine-tuning and nvidia-smi command-line utility to observe the GPU and memory utilization.

1.4 Scope of the Study

The scope of this research work is defined as follows.

- Only open-source LLM will be used for fine-tuning. Like Meta's Llama 2, Llama 3 (Touvron et al., 2023), Mixtral from Mistral AI, Falcon from tiuae (Almazrouei et al., 2023). Phi-1, Phi-2 from Microsoft, GPT-2 from OpenAI (Brown et al., 2020)
- This experiment will consider Performance-Efficient Fine-Tuning (Ding et al., 2023) methods like LoRA (Hu et al., n.d.) and QLoRA (Dettmers et al., 2023) to implement model fine-tuning.
- HuggingFace and other open-source software and libraries will be used for implementation.
- Either publicly available free GPUs such as Google Collab, Kaggle, or pay-per-use Amazon Web Services (AWS) SageMaker notebook will be used for implementing and running the LLM fine-tuning.
- The project will be completed within 17 weeks after the research proposal submission.

Out of scope

- Study of model hallucination (Huang et al., 2023b; Gekhman et al., 2024; Ravi et al., 2024) because of fine-tuning.

- Implementation of PEFT using other methods than LoRA and QLoRA, like Prefix Tuning (Lisa Li and Liang, n.d.), P-Tuning (Liu et al., 2021), Prompt Tuning (Lester et al., 2021), IA3
- Large Language Models, bigger than 2 billion parameters will not be considered for fine-tuning because of hardware and cost limitations.

1.5 Significance of the Study

Since the emergence of Large Language Models, the domain of Natural Language Processing has transformed because of LLM's advanced and highly sophisticated capabilities. These language models are pre-trained on large numbers of datasets and, can perform various range of tasks including but not limited to Text Translation, Text Generation, Summarization, and Question-Answers. Although LLMs perform well at doing generic language tasks they perform poorly at accomplishing the tasks in specialized fields or domains.

LLM fine-tuning helps to turn the generalized pre-trained LLM into performing more specialized tasks. By fine-tuning the model on specialized datasets, the performance of the model in that domain is greatly improved while preserving the original knowledge, thus fixing the issue of catastrophic forgetting. (Goodfellow et al., 2013)

LLM fine-tuning is a data-intensive job and it requires a large volume of high-quality labeled datasets. In practice all the specialized datasets required for model fine-tuning might not be available at once, it might be available at regular intervals and in batches. So, deciding the frequency of LLM fine-tuning and the amount of data to be used is crucial, as fine-tuning is a resource, time-intensive, and costly process. Datasets required for fine-tuning can be created manually or using computer programs.

This research tries to find out the effect of frequent fine-tuning using smaller datasets versus infrequent fine-tuning using larger datasets on LLM performance and resource requirements. This research will guide AI Engineers and Data Scientists to decide on the frequency and amount of data to be used for optimal model performance and resource utilization.

1.6 Structure of the Study

The structure of the study is as follows:

- Chapter 1, Introduction: The introduction and background of the research work will be covered in this chapter.
- Chapter 2, Literature Review: This chapter contains related works about the LLM fine-tuning methods and practices.
- Chapter 3, Methodology: This chapter provides details about the methodology and algorithms used during the experimentation stage.
- Chapter 4, Implementation: This chapter contains the implementation details of various experiments performed for implementing the model fine-tuning.
- Chapter 5, Results: Results of the experiments performed in Chapter 4 are mentioned in this chapter.
- Chapter 6, Conclusion: Conclusions and results of the experiments are mentioned in this chapter along with the future improvements.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

This chapter discusses some of the important work done in the Large Language Model fine-tuning field.

2.2 Transformer Architecture

2.1.1 Introduction

What is common among OpenAI's Chat GPT, Google's Gemini and BERT (Devlin et al., n.d.), Meta's LLaMA (Touvron et al., 2023), and all other famous LLMs? All these modern Generative AI Large Language Models are based on the new neural network architecture introduced by a team at Google Brain and a group from the University of Toronto in 2017 called Transformer (Vaswani et al., 2017). The Google-introduced attention mechanism is the heart and soul of Transformer architecture. Since its introduction, Transformer has become the go-to architecture for machine translation and other Natural Language Processing tasks like Question-Answering, Summarization, and Text Prediction.

Before transformer architecture, Recurrent Neural Network (RNN) and its variants like Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) were famous for designing and developing solutions for sequential data processing tasks. However, these RNN-based traditional models could not capture the long-range dependencies and thus had a short context. Also because of the limited context it was difficult for these models to figure out how words placed far apart in the sentence might be related to each other.

2.1.2 Attention Mechanism

The attention mechanism introduced by Google is the heart and soul of the transformer Neural Network architecture. Traditional models treat each word in the input sentence independently but the attention mechanism allows transformers to assign different weights to different words of the sentence based on their importance. Because of the attention mechanism model can capture the long-range dependencies, resolve the ambiguity, and understand the nuances more effectively.

Consider the following two sentences.

1. The **leg** of the chair is broken, please use a different chair
2. I hurt my **leg** during the football practice this morning.

The traditional embedding method will assign the same vector to the word **Leg** in both sentences. Because of the same vector representation, the model will not be able to distinguish the meaning correctly.

The attention mechanism solves this issue by analyzing the surrounding words and calculating the attention score to determine the relevance. This attention score is then used to weight the embedding vector representation, resulting in different vector representations for the word **Leg** as the chair part in the first sentence and **Leg** as the human limb in the second sentence.

2.1.3 Transformer Model

The encoder and decoder are the two main components of the transformer (Vaswani et al., 2017) based models.

2.1.3.1 Encoder and Decoder

Encoder:

The encoder contains six identical layers placed on top of each other. Each layer contains one multi-head self-attention layer and one position-wise fully connected feed-forward network. A residual connection is employed around each of the two sub-layers followed by layer normalization (Ba et al., 2016).

Decoder: Similar to the encoder, the decoder also contains six identical layers placed on top of each other. Along with the two encoder layers, to perform the multi-head attention over the output of the encoder, the decoder inserts the third sub-layer. Similar to the Encoder, residual connection is employed around each of the two sub-layers followed by layer normalization. Self-attention sub-layer in the decoder is modified to prevent positions from attending to subsequent positions.

2.1.3.2 Attention

The attention function is a mapping of a query and set of key-value pairs to an output. Here query, keys, values, and output all are vectors. The output is the weighted sum of all the values.

Weights assigned to each value are calculated by the compatibility function of the query with the respective key.

2.1.3.2.1 Scaled Dot-Product Attention

The input to the Scaled Dot-Product Attention is queries and keys of dimension d_k , and values of dimension d_v . The Dot product of the query is computed with all keys, divided by the square root of d_k and finally softmax function is applied to the values to get the weights.

The attention function is computed on a set of queries simultaneously as matrix Q . Keys and Values are also the matrices K and V .

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \text{Sqrt } d_k)V$$

2.1.3.2.2 Multi-Head Attention

The multi-head attention mechanism uses multiple scaled dot-product attention head in a parallel fashion. Each attention head has its own query, key, and value matrices. This model attends to different parts of the input in different ways.

The output of the individual attention head is concatenated with the output of multi-head attention. This makes the model to learn a richer representation of the input.

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head1}, \dots, \text{headh})W^O \\ \text{where head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^k) \end{aligned}$$

2.1.3.2.3 Position-wise Feed-Forward Networks

Along with the attention sub-layer, every layer in the encoder and decoder contains a fully connected feed-forward network. This feed-forward network is separately and identically applied to each position. Feed-forward network contains two linear transformations with a ReLU activation. It allows an introduction of non-linearity in the linear attention calculation.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

2.1.3.2.4 Embeddings and Softmax

Embeddings are used to convert the input and output tokens to vector representations. Softmax function is used to convert the decoder output to predict the probability of the next token.

2.3 Pre-Trained Language (PTL) Model Fine-Tuning

2.3.1 Introduction

A pre-trained Machine Learning (ML) model is trained on a large volume of generalized datasets and can perform a variety of Natural Language Processing (NLP) tasks such as Text Summarization, Classification, Machine Translation, and Question-Answer. These pre-trained models can be used to accomplish a specific task in specialized domain or field by fine-tuning the model with domain-specific dataset. Pre-trained models are an efficient and effective way for developing specialized Machine Learning models as they are already trained on large volumes of data and provide an initial set of weights and biases.

In the model, fine-tuning (Wei et al., 2021) these initial weights and biases are further trained on additional domain-specific data to make an LLM expert in accomplishing more specialized tasks. Model fine-tuning leverages the concept of Transfer Learning. In Transfer Learning knowledge learned for one task is re-used to accomplish the other similar tasks. It helps to save time and resources required to re-train the entire model.

As these general-purpose language models are trained on vast amounts of data, they tend to be very large. Generally, smaller LLM contain a few billion parameters on the other side larger LLM contains a couple of hundred billion parameters. The smallest falcon (Almazrouei et al., 2023) LLM built by tiuae has around 1 billion parameters and the largest one has 180 billion parameters. Fine-tuning LLM requires expensive GPUs and a considerable amount of time.

2.3.2 Parameter Efficient Fine-Tuning (PEFT)

Transformer architecture based Large Language Models (LLMs) like GPT-3 (Brown et al., 2020), BERT (Devlin et al., n.d.), and Llama (Touvron et al., 2023), have shown tremendous capabilities and mind-blowing results in Natural Language Processing task. Conventionally these models are pre-trained with generic publicly available internet data and then fine-tuned for some specific task or domain. As compared to out-of-the-box pre-trained models, fine-tuned (Ding et al., 2023) models perform far better when it comes to specialized downstream tasks.

As the model gets larger, full parameter fine-tuning cannot be done on consumer hardware. Also deploying fine-tuned models for every independent task becomes very expensive as the size of the fine-tuned model is the same as the original pre-trained model. Parameter-Efficient Fine-Tuning (Ding et al., 2023) is the answer for both of the issues mentioned above.

Parameter-Efficient Fine-Tuning (Ding et al., 2023) methods such as Low-Rank Adapters (LoRA) (Hu et al., n.d.), Quantized Low-Rank Adapters (QLoRA) (Dettmers et al., 2023), Prefix Tuning (Lisa Li and Liang, n.d.), AdaLoRA , Prompt Tuning (Lester et al., 2021) provides a way to fine-tune these large models with limited resources and time. As the resource and time required for PEFT is drastically reduced compared to full parameter fine-tuning (Lv et al., 2023) even smaller companies and universities can also fine-tune the LLMs for specialized downstream tasks and participate in the research and development activities.

PEFT only fine-tunes the smaller subset of the model parameters while keeping most of the other pre-trained model parameters frozen. As the number of parameters participating in the fine-tuning process decreases, the resources and time required for the fine-tuning process also decrease proportionally. PEFT also helps to overcome the issue of catastrophic forgetting (Goodfellow et al., 2013). The performance of the LLM trained using the PEFT method is comparable with the full fine-tuned LLM although PEFT only trains a small number of parameters.

2.3.2.1 Low Rank Adaptation (LoRA)

A major challenge in LLM fine-tuning is the amount of time and resources required. As model size increases, the number of parameters required to be updated or fine-tuned also increases proportionally. This makes regular fine-tuning of models a very expensive and time-consuming process. If a model is not fine-tuned regularly it is most likely to become absolute after some time. So regular Fine-tuning of the model with the latest datasets is necessary to keep the model up to date with the latest information.

The pre-trained large language models have a low intrinsic dimension (Aghajanyan et al., 2020) and they can learn efficiently even though a smaller number of model parameters are updated. This concept is further explored in Low-Rank Adaptation, which updates the model weights and also has a low intrinsic rank during fine-tuning.

The Low-Rank Adaptation method keeps the original pre-trained model as it is and adds a small updatable part to each layer of the model. As the number of trainable parameters reduces the resources and time required for the model fine-tuning process also reduces drastically. In turn, this reduces the model fine-tuning cost, making the model fine-tuning affordable.

For Example, GPT-3 has a total of 175 billion parameters and in full fine-tuning, all these 175 billion parameters need to be updated. On the other hand, using LoRA the trainable parameters will be reduced around 10K times, and three times less GPU will be required.

In implementation, the number of parameters that need to be updated during fine-tuning can change based on the rank of the LoRA matrix decided during the implementation. The higher the rank more the parameters are to be updated during fine-tuning and vice versa. Higher-rank models will take more resources and longer time to complete the fine-tuning and may perform better than low-rank model fine-tuning. Thus it's extremely important to strike the perfect balance between model accuracy and LoRA rank to achieve the highest quality and low fine-tuning cost.

Currently, there is no framework or rule to select the weight matrices, to which LoRA should be applied. Selecting smaller rank matrices will reduce the trainable parameters but may not perform well on the other hand selecting larger rank matrices will increase the trainable parameters but may be more performant. Further, the study is required to select the right rank of the matrix to strike the balance between training cost and model performance.

2.3.2.2 Quantized Low-Rank Adaptation (QLoRA)

Large Language Models are fine-tuned to improve their performance and add more capabilities to the model. Full fine-tuning of the LLM requires a large number of expensive GPUs and it's also time-consuming. Low Rank Adaptation (LoRA) (Hu et al., n.d.) method of the Parameter-Efficient fine-tuning approach, reduces the trainable parameter.

QLoRA (Dettmers et al., 2023) can be called an advanced version of LoRA, it works by quantizing the precision of model parameters in the pre-trained language models to 4-bit precision. Generally model parameters are stored in 32-bit format, but QLoRA converts them to 4-bit format. Mathematically this reduces the model size roughly by 8 times, without sacrificing performance. As the memory requirement of LLM is reduced, now it is possible to run the LLM on less powerful hardware.

Normally a 65B parameter model needs on average more than 780 GB of GPU memory for fine-tuning. With QLoRA less than 48 GB of GPU memory is required for fine-tuning the same 65B model without degrading the model runtime or performance when compared with a 16-bit fully fine-tuned baseline.

QLoRA has introduced many innovative ways to reduce the memory footprints of the LLMs.

1. QLoRA has introduced a new 4-bit NormalFloat datatype. Theoretically, this datatype optimally quantizes the normally distributed data and has better results than 4-bit float and 4-bit integers.
2. The double quantization method quantizes the quantization constant and saves around 0.37 bits per parameter on average.
3. Paged Optimization is a way of avoiding gradient checkpointing memory spikes using NVIDIA unified memory. Paged optimization prevents memory spikes during gradient checkpointing, thus allowing the LLM to be fine-tuned on a single GPU.

Currently, QLoRA can only be implemented with the LoRA. Many other Parameter-Efficient Fine-Tuning methods need to be explored. Model fine-tuning performed after the quantization seems to recover most of the details lost during the quantization, this could potentially enable much more aggressive quantization. For example, 3-bit or 2-bit quantization will further reduce the memory required for loading the model and make the fine-tuning process even more affordable.

2.3.2.3 Prefix Tuning

Prefix Tuning (Lisa Li and Liang, n.d.) is a lightweight Parameter-Efficient Fine-Tuning (PEFT) alternative to fine-tuning the natural language generation task. Prefix Tuning optimizes the sequence of continuous task-specific vectors called prefix and keeps the LLM parameters frozen.

In Prefix Tuning sequence of continuous task-specific vectors is prepended to the input, which is called a prefix. To generate each token, the LM can attend to the prefix as if it were a sequence of “virtual tokens”, but unlike prompting, the prefix consists entirely of free parameters which do not correspond to real tokens.

In fine-tuning all the LLM parameters are updated and a fine-tuned version of the model for each task is stored, on the other hand, prefix tuning only optimizes the prefix, resulting in a very small overhead for each additional task.

The number of trainable parameters increases proportionately with the prefix length. More trainable parameters and more computing power are required resulting in higher training costs. Performance of the model increases as the prefix length increases up to certain threshold. Performance drops slightly once the prefix length breaches the threshold because of overfitting of the training data.

The prefix tuning method of PEFT is only tested against text summarization and table-to-text tasks. Further research is required to test this approach for other Natural Language Processing tasks like Classification, Question-Answer, Text Completion, and Text Generation, Machine Translation.

2.3.2.4 Prompt Tuning

Prompt Tuning (Lester et al., 2021) is a simple yet effective mechanism for learning “soft prompts” to condition frozen language models to perform specific downstream tasks. Soft Prompts are learned through backpropagation and can be tuned to incorporate signals from any number of labeled examples.

Prompt Tuning is a simplified version of Prefix Tuning (Lisa Li and Liang, n.d.) PEFT approach. In Prompt Tuning, some vectors are prepended at the beginning of a sequence at the input layer. The embedding layer converts each token of the input sentence to its corresponding word embeddings, and the prefix embeddings are prepended to the sequence of token embeddings. Next, the pre-trained transformer layers will process the embedding sequence like a transformer (Vaswani et al., 2017) model does to a normal sequence. During the fine-tuning process, only prefix embeddings are updated while keeping the rest of the transformer model frozen and unchanged.

Prompt Tuning is a competitive approach for adapting frozen pre-trained Large Language Models to specialized downstream tasks. On the SuperGLUE performance benchmark, its task performance is comparable with traditional model tuning. As the model size increases Prompt Tuning performance gap also closes. On zero-shot domain transfer, Prompt Tuning (Lester et

al., 2021) leads to better generalization. This possibly indicates that freezing general-purpose language understanding parameters and restricting downstream learning to a lightweight parameter footprint could avoid overfitting to a specific domain.

2.4 Distributed Parameter-Efficient Fine-Tuning (DLoRA)

LLM fine-tuning is a resource-intensive task and it needs several GPUs. Usually, organizations use cloud service providers like AWS, Azure, or GCP to rent the GPUs as and when required. User data is shared with the public cloud services over the internet, this raises potential privacy concerns about the user data.

Distributed Parameter-Efficient Fine-Tuning DLoRA (Gao and Zhang, 2024) enables scalable PEFT operations to be performed collaboratively between the public cloud platforms and user devices. The Kill & Revive algorithm used by DLoRA helps to significantly reduce the communication and computation workload over user devices and achieves higher accuracy and privacy protection.

In the Cloud-Only solution, the PEFT solution is deployed on a cloud server and all the necessary user input is also transferred online. Storing private user data in a shared cloud environment has major security and privacy risks. Another option is to perform an LLM fine-tuning process on the user device. This is called an Edge-only solution, but this is impractical most of the time because the user devices have limited computing power and storage.

Distributed PEFT solution DLoRA (Gao and Zhang, 2024) is a collaborative operation between the cloud and the user device. In DLoRA the private user data required for LLM fine-tuning and the personal LLM parameters are stored on the user device, so there is no need to transfer the data to the cloud. Thereby it minimizes the risk of privacy leakage. DLoRA also offloads some LLM fine-tuning computation to user devices to mitigate the scalability issues.

The kill and Revive (KR) algorithm dynamically identifies and fine-tunes some of the LLM parameters very sensitive to the training data. This method results in a notable decrease in the computational and communication workload on the user device. A minimal set of tuneable parameters are searched by the Early-Kill mechanism, thus eliminating the redundant fine-tuning operations with very little effect on accuracy.

When compared with both edge-only and cloud-only solutions, DLoRA requires less computation and communication overheads without compromising accuracy and privacy.

2.5 Multiple-Trained LoRA Composition

Recently some initiatives have explored the composition of separately trained multiple LoRAs to achieve a combined generation of learned capabilities. (Huang et al., 2023a) Linear Arithmetic composition (Zhang et al., 2023) directly composes the trained LoRAs. A generative performance of pre-trained models is degraded if three or more LoRAs are composed together. These performance issues can be resolved by applying weight normalization before the composition. But this can affect the unique attributes of each trained LoRAs as composing weight is reduced. Reference tuning-based composition which is specifically designed for the V&L field demonstrates better performance. However, this approach has limited LoRA flexibility because of the utilization of manually designed masks and higher training costs, requiring a full model re-training.

Mixture of LoRA Experts (Wu et al., 2024) MOLE, recognizes that individual layers of a trained LoRA showcases different attributes, which collectively showcases the overall attributes of the trained LoRA. MOLE modulates the weights of different trained LoRA in each layer called as “ hierarchical weight contro”. MOLE considers each layer of trained LoRA as an individual expert and implements a gating feature within each layer to learn the optimal composition weights based on the specified domain. This dynamically increases desirable characteristics while mitigating less favorable ones, ultimately achieving a more efficient composition of LoRAs and avoiding the loss of required LoRA characteristics.

MOLE learns the gating function for multiple trained LoRAs during training and keeps all other parameters unchanged, thus saving computational costs. MOLE has two inference modes.

- MOLE utilizes all trained LoRAs with the learned gating function, preserving individual characteristics with allocated weights.
- Allows masking of unwanted LoRAs and re-calculates and distributes weights proportionally without the need for re-training.

The above two modes allow MOLE to adapt to different scenarios, providing a flexible and versatile approach for effective LoRA composition.

The performance of all the MOLE decreases when the number LoRAs are increased to a large value.

For example 128. MOLE fails to perform large-scale LoRA composition. Further research is required for better handling of large-scale LoRA compositions effectively.

2.6 LLM Fine-Tuning and Hallucination

LLM hallucination (Gekhman et al., 2024) is a phenomenon where LLM generates grammatically and syntactically correct but factually incorrect and non-existent answers. LLM hallucination is one of the main obstacles in the practical deployment of LLMs to production and raises concerns over the reliability of LLMs in production scenarios. Because of this, it's extremely important to detect and mitigate the model hallucination. If not addressed properly there could be dire consequences if output of LLM is used as it is for the practical use cases. In one such example, a lawyer in the United States of America (USA) used ChatGPT for legal research and submitted the ChatGPT-generated answers without any confirmation or validation. Because of the model hallucination ChatGPT presented non-existent and factually incorrect answers, and both the lawyer and law firm were fined USD 5000 by the USA court. In another case, Alphabet's (Google's parent organization) market capitalization dropped around USD 100\$ when Google's LLM generated the incorrect fact in the promotional video posted on X (formerly known as Twitter)

A survey was conducted to redefine the taxonomy of hallucination (Huang et al., 2023b) and to create a more robust framework for LLM applications.

2.6.1 Types of LLM Hallucination

LLM hallucination can be broadly divided into the following two types.

2.6.1.1 Factuality Hallucination

Responses provided by the model are factually incorrect and non-existent. In short, the model gives made-up answers. Factuality hallucination further can be divided as follows.

- **Factual Inconsistencies:** This refers to the situation where content generated by the model is verifiable with real-world facts and information. When verified model responses turn out to be wrong.

- **Factual Fabrication:** It refers to the situation where content generated by the model is not verifiable with real-world facts and information.

2.6.1.2 Faithfulness Hallucination

Model response is different than the instruction given by the user and the response is contextually different. Faithfulness hallucination can be further divided as follows.

- **Instruction Inconsistency:** LLM output is different than the instruction provided by the user.
- **Context Inconsistency:** LLM's output is unfaithful with the user-provided contextual information.
- **Logical Inconsistency:** LLM's output shows internal logical contradiction, mostly seen in reasoning tasks.

2.6.2 Causes of Hallucination

Hallucination in LLM can have multiple origins, right from data gathering to model inference in production.

2.6.2.1 Hallucination from Data

Pre-training data is the basic foundation of the LLM, it makes them capable of gaining knowledge and factual details. However flawed data sources and poor utilization of factual knowledge captured in data can contribute to hallucination.

2.6.2.2 Hallucination from Training

LLM training mainly consists of two stages; 1) the pre-training stage; where LLM learns general-purpose representation and captures the factual knowledge present in the data, and 2) the alignment stage; where LLM is adapted to better align with user preferences and instructions. These two processes add incredible capabilities to the LLM but any mistakes in these stages can lead to hallucination.

2.6.2.3 Hallucination from Inference

Decoding plays an important role in manifesting the capabilities of the LLM, after pre-training and alignment. However, any mistakes during the decoding strategy can lead to LLM hallucinations.

2.6.3 Model Fine-Tuning and Hallucination

A controlled study was conducted to analyze the effect of new knowledge on the fine-tuned LLM, using a closed-book question-answer pattern. (Gekhman et al., 2024) The proportion of fine-tuning examples introducing new knowledge was changed while other factors were controlled.

LLM fine-tuning examples were separated into two types. “Known” and “Unknown”. Known fine-tuning examples were further divided into subcategories. “HighlyKnown”, “MaybeKnown” and “WeeklyKnown”. This process of selecting the test samples is known as SliCK (Sampling-based Categorization of Knowledge).

The research details and the findings are as follows

- Fine-tuning examples consistent with the model’s pre-existing knowledge are learned significantly faster than the fine-tuning examples introducing new knowledge.
- LLM’s tendency to hallucinate concerning its existing knowledge increases as it learns from new knowledge.
- Utilization of pre-existing knowledge by LLM is significantly influenced by the composition of Known examples in the fine-tuning data.

LLM mainly gains factual knowledge or information during pre-training, on the other hand, fine-tuning improves LLM’s capability to utilize the knowledge or information learned during pre-training. The introduction of new knowledge or information during fine-tuning can increase the tendency of LLM to hallucinate.

2.7 Hallucination Evaluation

Retrieval Augmentation Generation (RAG) helps to overcome the hallucination in LLM by introducing more detailed data, for example, the latest updates and even private data to the LLM. As the model gains more knowledge about certain domains, theoretically it should produce factually correct outputs without hallucination. But practically in real time model can

still provide factually incorrect answers leading to model hallucination. It's very important to evaluate model hallucinations before deploying them to production. LYNX (Ravi et al., 2024) is a state-of-the-art LLM that can detect hallucinations, and it is also capable of advanced reasoning on challenging real-world hallucination scenarios. HaluBench was used to evaluate the LYNX. It is a comprehensive hallucination evaluation benchmark and it consists of around 15k samples collected from multiple real-world domains. LYNX performs better than Claude-3-Sonnet, GPT-4o, and other open and closed-sourced LLM-as-a-judge models on the HaluBench benchmark.

LYNX (70B) performs better than GPT-4o and closed-source LLMs in hallucination detection tasks and it is also the first open-source hallucination detection model that performs better than GPT-4o and closed-source LLM-as-a-Judge. LYNX is a fine-tuned version of the Llama-3-70B-Instruct model. Data from multiple Question Answer (QA) datasets such as CovidQA, PubmedQA, DROP, and FinanceBench is used to generate hallucinated answers that seem plausible but are not faithful to the context. The model is considered as hallucinating if the answer provided by LLM for the given question is not supported by the context.

RAG systems are gaining popularity as they provide a fast and efficient way to introduce private, detailed data to the LLM. For safe deployment of RAG to production an automated evaluation of RAG becomes necessary for scaling purposes. LYNX is an evaluation model that accesses the faithfulness of model output without any reference. HalluBench is a comprehensive evaluation benchmark and it contains annotations of faithfulness of model output across multiple real-world domains. HalluBench contains an equal distribution of negative and positive examples, this makes it unique from other benchmarks.

Currently, HalluBench only supports English language Question-Answering model evaluation. Further work is needed to support multi-lingual hallucination evaluation. Support to evaluate hallucination in other NLP tasks like Summarization, Text Generation, and Translation is not supported by HalluBench, the current version only concentrates on the evaluation of the Question-Answer based LLMs.

2.8 Summary

There has been tremendous work done in the development of Large Language Models and fine-tuning of these models for more specific downstream tasks. Both model pre-training and fine-tuning are resource and time-intensive tasks. With the emergence of Parameter-Efficient Fine-Tuning methods like Low-Rank Adapter (LoRA) (Hu et al., n.d.), QLoRA (Dettmers et al., 2023), Prefix Tuning (Lisa Li and Liang, n.d.). etc both resources and time required for model fine-tuning have reduced many folds as these methods do not fine-tune the whole model but just adjust a few required parameters and keep the rest of the model frozen.

The tests have proved that there is no significant performance loss with PEFT methods when compared with the full parameter fine-tuning. This makes PEFT a very popular and cost-effective solution for model fine-tuning. Even small enterprises and universities can do research with model fine-tuning by adopting PEFT methods.

It's important to address the issue of model hallucination as it affects the production adaptation of LLM. Hallucination can bring trust issues with the LLM if not addressed correctly.

CHAPTER 3: RESEARCH METHODOLOGY

3.1 Introduction

This research focuses on fine-tuning the Large Language Models (LLMs) using Parameter-Efficient Fine-Tuning techniques. Some of the concepts, algorithms, and techniques required for LLM fine-tuning, LLMs, and datasets are discussed in this chapter.

3.2 Large Language Models & PEFT Techniques

3.2.1 Large Language Model (LLM)

Large Language Models or Pre-Trained Models (PTM) are a type of artificial intelligence algorithm that uses deep learning techniques to understand and generate human language. Most of these modern LLMs are based on transformer architecture introduced by Google in 2017 in the **Attention is All You Need** research paper. The transformer is a novel neural network architecture consisting of encoder and decoder along with self-attention capabilities. The encoder and decoder are used to understand the meaning of the sentence and also understand the relationship between words and phrases in the sentence.

LLMs are pre-trained with vast volumes of data using specialized hardware called GPUs. LLM training is an expensive and resource-intensive process. Typically LLM contains billions of parameters. Such huge LLMs are trained with massive amounts of publicly available internet data. According to the internet, it took 34 days to train the GPT-3 175B model using 1024 Nvidia V100 GPUs and cost around USD 4.6 million.

LLMs are very efficient and effective in performing Natural Language Processing tasks like Text Generation, Text Completion, Summarization, Classification, Translation, and Question-Answer. Gemini from Google, Llama from Meta, and ChatGPT from OpenAI are some of the well-known real-life examples of LLM.

3.2.2 LLM Fine-Tuning

Large Language Models are pre-trained on vast amounts of public internet data and they can perform well on general purpose Natural Language Processing tasks. The model gains a general understanding of the language and its structure during the pre-training, but it's not enough for

the model to perform some specialized domain-specific tasks. Due to this performance of these LLMs is not so good or very limited while performing the NLP tasks in some specific domains. e.g medical, science, legal, or sports.

LLM fine-tuning is the most common method to enable general-purpose models to perform downstream specialized tasks. Fine-tuning enables the adaptation of LLMs for more specialized tasks and domains. Fine-tuning helps in the practical adaptation of LLM for specialized business use cases where LLM is fine-tuned with private data. If LLM is fine-tuned with task-specific datasets, then the performance of the LLM will be improved on that specific task and domain. Also, the knowledge learned during the pre-training phase is preserved.

A brief process of LLM pre-training and fine-tuning is illustrated in below mentioned diagram. A pre-trained model is fine-tuned with the custom dataset to get the fine-tuned LLM.

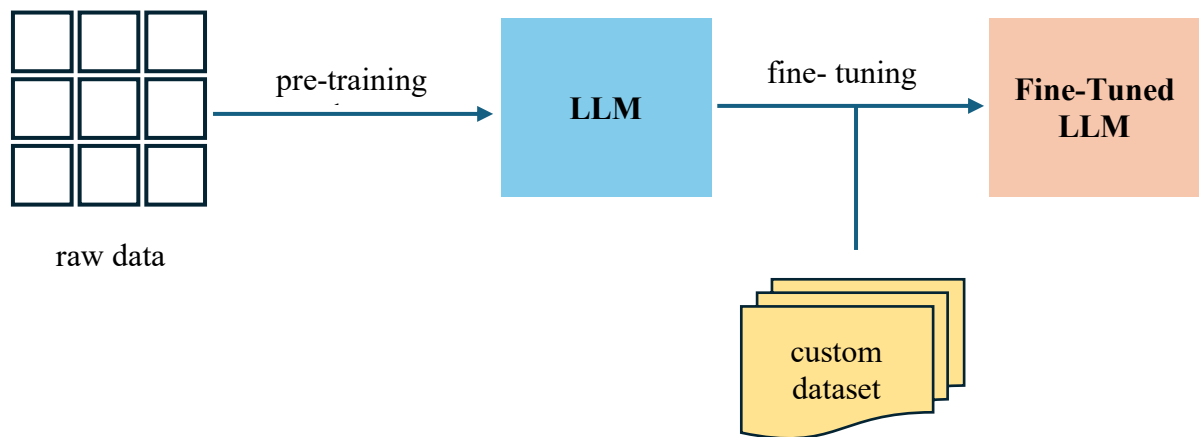


Figure 1: LLM Pre-Training and Fine-tuning Process

3.2.3 Types of LLM Fine-Tuning

Similar to model pre-training, fine-tuning is also an expensive and time-consuming task. It also needs specialized hardware called GPU for running the model fine-tuning jobs. In recent times massive research has happened around LLM fine-tuning techniques and approaches to make it affordable and faster. Some of the popular fine-tuning methods are described below.

3.2.3.1 Full Parameter Fine-Tuning

As the name suggests; the full parameter LLM fine-tuning updates all the weights and biases of the model neural network. The entire model is trained on task or domain-specific datasets, resulting in adjustments to all model layers. Full parameter fine-tuning can potentially result in better model performance compared with other fine-tuning methods. Memory and computation power required for full parameter fine-tuning is significantly higher, sometimes even the same as pre-training. As all the model weights and biases are updated, there are chances of catastrophic forgetting in Full parameter fine-tuning.

3.2.3.2 Parameter-Efficient Fine-Tuning

Model fine-tuning helps to achieve transfer learning. Parameters of the pre-trained model are further trained on new data to understand new trends and patterns. Training an LLM is a computationally extensive task and takes a considerable amount of time. Memory is required not only to store the model but also the parameters that are necessary for the model training process. For simple hardware, it's not possible to handle such memory and computationally demanding tasks.

This is why PEFT is important. Full LLM fine-tuning updates all the model parameters on the other hand PEFT method only updates a smaller subset of model parameters keeping most of the other parameters untouched. Typically, only 10-15% of the model parameters are updated by PEFT in the model fine-tuning process, keeping the rest of the parameters frozen or untouched.

Because of PEFT, computation, and storage required for model fine-tuning are reduced drastically thus lowering the overall cost of fine-tuning significantly. PEFT not only reduces the computational and storage requirements but also deals with catastrophic forgetting. As it does not change the original LLM, the model does not forget the previously learned information during the pre-training phase.

3.2.3.2.1 Low Rank Adaptation (LoRA)

LoRA introduces a new additional layer of trainable parameters for model training while keeping the original model weights frozen. These additional parameters are of reduced rank (Dimensions) compared to the dimensions of the original network. This reduces the number of trainable parameters significantly, thus reducing the computation and storage requirements.

Consider there are 256×256 trainable parameters in the feed-forward network, making the total number of trainable parameters equal to 65,536. These original parameters are frozen during fine-tuning and the LoRA adapter with rank 2 is added. So now, the new training parameters are 512×2 for WA and 512×2 for WB. Thus, the total trainable parameters are $WA\ 1024 + WB\ 1024 = 2048$. These new 2048 parameters are fine-tuned with domain-specific data.

Finally, these new parameters are multiplied with the original frozen layer once the fine-tuning is completed.

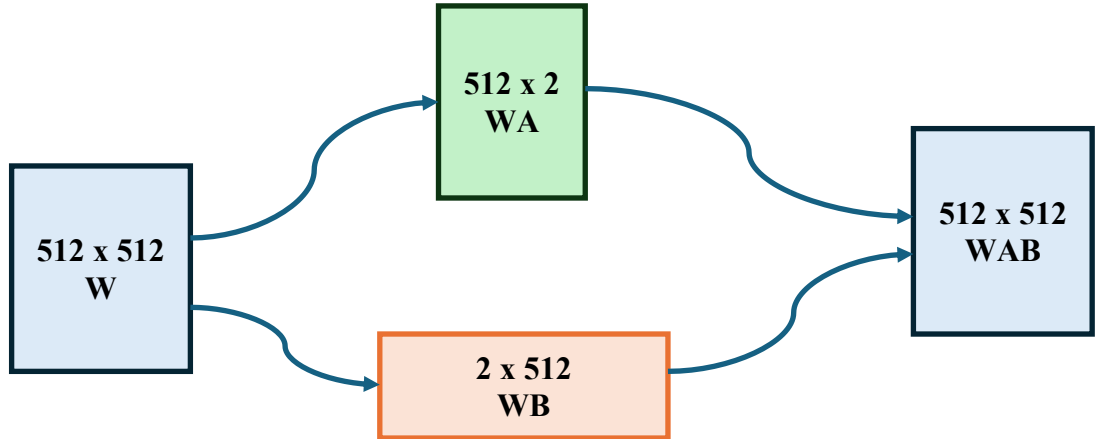


Figure 2: Parameter-Efficient Fine-Tuning using LoRA

3.2.3.2.2 Quantized Low-Rank Adaptation (QLoRA)

QLoRA can be called an advanced version of LoRA, it requires even less computing and storage than LORA. QLoRA takes a LoRA one step ahead by quantizing the weights of LoRA adapters (smaller matrices) to lower precision. e.g. 4 bits instead of 8 bits used by LoRA. This further reduces the memory required and storage needed almost by half. In QLoRA the pre-trained model (LLM) is loaded into GPU memory with quantized 4-bit weights, in contrast to 8-bits used in LoRA. Even though the bit of precision is reduced in QLoRA, it does not affect

the effectiveness or accuracy of the model significantly. The effectiveness and accuracy of QLoRA models are comparable with the effectiveness and accuracy of LoRA fine-tuned models.

With QLoRA it is possible to fine-tune a large 65B parameter model on a single GPU with just 48GB memory, without any loss in quality compared to full 16-bit training. Additionally, QLoRA makes it feasible to fine-tune large models with full 16-bit precision on standard academic setups, paving the way for more exploration and practical uses of Large Language Models (LLMs).

3.3 Transformer Based Language Models

Most of the modern Large Language Models are based on the Transformer architecture. This architecture was introduced by the team at Google Brain in December 2017 in the **Attention is All You Need** research paper. Since transformer architecture is better than Recurrent Neural Network (RNN), Long Short Term Memory (LSTM), and Gated Recurrent Unit (GRU) for Natural Language Processing tasks, it has become widely popular to build Natural Language Processing language models.

Below listed models will be used for the implementation purpose, because of their smaller parameter size and low resource requirement for fine-tuning.

3.3.1 Falcon-RW-1B

Falcon-RW-1B is an open-source LLM designed and trained by the Technology Innovation Institute of the United Arab Emirates (tiiuae). Falcon-RW-1B is the smallest LLM in the Falcon model series. It is a casual decoder-only model and contains around 1.3B parameters trained on 350 billion tokens. RefinedWeb dataset is used to train the Falcon-RW-1B. This model is intended to be used for research purposes because of its smaller size and parameters. Only English language data is used to train the Falcon-RW-1B and it won't be generalized to other languages. 32 Nvidia A100 40GB GPUs were used to train the Falcon-RW-1B using data parallelism with ZeRO.

Other models in the Falcon series are listed below.

- Falcon-7b
- Falcon-rw-7b

- Falcon-7b-instruct
- Falcon-11b
- Falcon-11b-vlm
- Falcon-40b
- Falcon-40b-instruct
- Falcon-180b
- Falcon-180b-chat

3.3.2 Phi-1

Phi-1 is a family of small language models developed and trained by Microsoft. It is a transformer-based model with around 1.3 billion parameters. It is mainly used to generate basic Python code. Phi-1 is trained on various data sources, like bigcode/the-stack (huggingface), Q&A contents from StackOverflow, and other Python coding examples. The accuracy of Phi-1 is slightly over 50%, so the code generated by the model should be used for reference only. Generation of production-grade code is out of the scope of this model, so users should practice caution before using the generated code by the model. Phi-1 was trained using 8 Nvidia A100 40GB GPUs for 6 days.

3.3.3 Phi-1.5

Phi-1.5 is a successor model of Phi-1. Similar to Phi-1, Phi-1.5 is also a transformer-based model with around 1.3 billion parameters. In addition to the data used for training Phi-1, Phi-1.5 is also trained with a new data source containing various NLP synthesis tasks. Along with writing the Python code, Phi-1.5 can perform various NLP tasks like drafting an email, text summarization, and creating poems and stories. Contents generated by the model should be used for reference only. Phi-1.5 was trained using 32 Nvidia 40G GPUs for 8 days.

3.3.4 TinyLlama_v1.1

TinyLlama_v1.1 is a compact language model with around 1.1 billion parameters. The architecture and tokenizer of TinyLlama_v1.1 are the same as Llama 2. Because of this, it can be easily plugged into many open-source projects built on Llama 2. TinyLlama is ideal for applications that have limited computation and memory. Other models in the TinyLlama family are listed below.

- TinyLlama_v1.1: Standard version for general-purpose use cases
- TinyLlama_v1.1_Math&Code: Ability to understand code and solve math problems
- TinyLlama_v1.1_Chinese: Ability to understand Chinese language

3.3.5 GPT-2

GPT-2 (Brown et al., 2020) language model is based on transformer architecture and it is pre-trained on a very large corpus of English data using a self-supervised training method. All GPT family models are designed and trained by OpenAI. No labeled text or data was used to train the GPT-2 model. GPT-2 was trained to predict the next word in the sentence. Training data for GPT-2 was collected by scraping all the outbound links on Reddit. All the Wikipedia pages were removed from the training data, so this model is not trained on the Wikipedia pages. This is the smallest model in the GPT family with 124M parameters. Other models in the GPT family are listed below.

- GPT-Medium: 335M parameter version of GPT-2 model
- GPT-Large: 774M parameter version of GPT-2 model
- GPT-XL: 1.5B parameter version of GPT-2 model.

3.4 Methodology

Details of the methodology used for LLM fine-tuning are described in this section.

3.4.1 End-to-End Pipeline

Below mentioned flow diagram depicts the E2E pipeline of the LLM fine-tuning process.

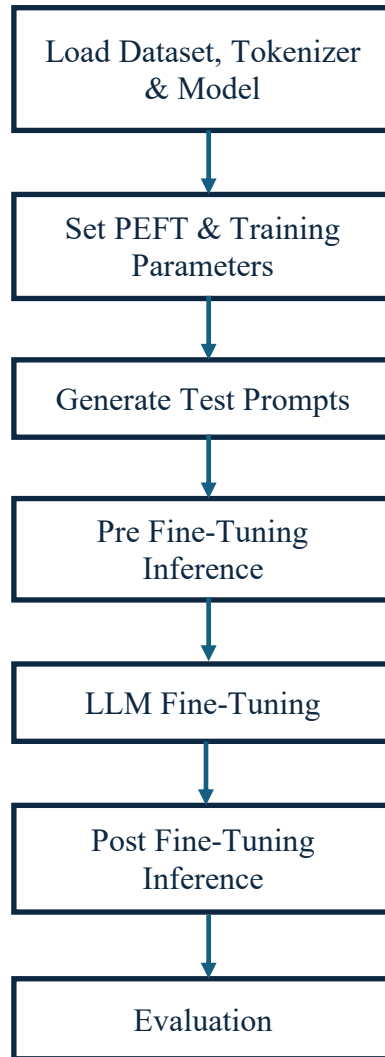


Figure 3: E2E LLM Fine-Tuning Pipeline

3.4.1.1 Load Dataset, Tokenizer and Model

The Dataset, Tokenizer, and Model need to be downloaded from huggingface using the datasets and transformer APIs provided by huggingface. Dataset will be used for model fine-tuning and Tokenizer will be used for tokenizing the input prompt for generating the model inference.

3.4.1.2 Set PEFT & Training Parameters

Setting PEFT parameters includes setting LoRA adapter rank, dropout rate, learning rate, batch size, etc.

Epoch count, training, and evaluation batch size, learning rate, logging steps, etc are configured as model training parameters. Further details about the Model Training hyperparameters are mentioned in Section 4.5

3.4.1.3 Generate Test Prompts

Few prompts are created to evaluate the model inference accuracy before and after fine-tuning. These prompts also help to evaluate the accuracy of different fine-tuning approaches. i.e. Full Dataset Fine-Tuning vs Split Dataset Fine-Tuning

3.4.1.4 Pre-Fine-Tuning Inference

LLM inferences using the prompts are generated in the above step before LLM fine-tuning.

3.4.1.5 LLM Fine-Tuning

LLMs are fine-tuning using the following two approaches.

- Full Dataset Fine-Tuning (FDFT)
- Split Dataset Fine-Tuning (SDFT)

3.4.1.6 Post-Fine-Tuning Inference

Model inferences are generated after the model fine-tuning is completed. Inference results obtained in pre-fine-tuning inferences are compared with post-fine-tuning inferences to analyze the improvement in accuracy. Also, the inference results from the Full Dataset Fine-Tuning approach are compared with the Split Dataset Fine-Tuning approach.

3.4.1.7 Evaluation

Both the LLM fine-tuning approaches i.e. Full Dataset Fine-Tuning and Split Dataset Fine-Tuning are evaluated based on the following parameters.

- GPU required
- Memory required
- Total Time taken
- Accuracy and performance

Zero-shot prompting approach will be used for generating the model inference. In the Zero-shot prompt technique, instructions are directly provided to the model without specifying any example.

e.g. Write an email to a close friend to congratulate them on getting married.

Nvidia provides the `nvidia-smi` command to retrieve the GPU and memory utilization details of the NVIDIA GPU. This command will be executed periodically to understand the memory and compute usage of GPU during the LLM fine-tuning process.

3.4.2 Datasets

Details of the datasets to be used for LLM fine-tuning are mentioned below.

3.4.2.1 microsoft/orca-math-word-problems-200k

This dataset was created by the Microsoft Research team in February 2024. This dataset contains around 200,000 school-level math work questions and answers. This dataset has been tailored to improve the mathematical capabilities of the LLM. Every data record in the dataset has a word math question and respective answer.

Data Fields:

The dataset has the following fields.

- Question: Mathematical question in string format
- Answer: Answer to the mathematical question

3.4.2.2 vezora/Tested-22k-Python-Alpaca

This dataset was released in November 2023 and consists of 22,000 working Python code samples. This dataset helps to build robust Python code LLM something like GitHub Co-Pilot but only concentrating on Python programming language.

Data Fields:

The dataset has the following fields.

- Instruction: Problem statement to write Python code.
- Input: An input to validate the Python code.
- Output: Python code for the given problem statement.

3.5 Tools

Hardware and Software tools required for the LLM fine-tuning are listed below. AWS SageMaker Jupyter Notebook instance will be used to run the LLM fine-tuning process.

3.5.1 Hardware

The following hardware resources will be required for the research.

- Laptop or desktop with internet access and software for writing/editing the documents.
- AWS SageMaker Notebooks with EC2 ml.g5.2xlarge instance (8 vCPUs, 32 GiB of RAM, 1 x NVIDIA A10G 24 GiB) designed for Machine Learning (ML) and other Graphics intensive processing

3.5.2 Software

The following software resources will be required for the research.

- Web Browser
- Jupyter Notebook
- Python 3.0+
- NVIDIA CUDA
- nvidia-smi
- Deep learning libraries. E.g. tensorflow, pytorch
- Python libraries for data analysis and visualization. E.g numpy, pandas, matplotlib, seaborn.
- Python libraries by huggingface for LLM fine-tuning implementation. E.g Bitsandbytes, transformers, peft, accelerate, datasets, einops
- Open Source LLM Models. E.g. phi-1, falcon-rw-1b, gpt-2, tiny_llama
- Amazon Web Services (AWS) Sage Maker

3.6 Summary

All the necessary LLM concepts, algorithms, fine-tuning approaches, and language models along with detailed research methodology are discussed in this chapter. This chapter also highlights the requirement of hardware and software tools necessary for the implementation of LLM fine-tuning.

CHAPTER 4: IMPLEMENTATION DETAILS

4.1 Introduction

Details about the implementation of LLM fine-tuning using the Full Dataset and the Split Dataset approach are discussed in this chapter.

4.2 Dataset Details

“Vezora/Tested-22k-Python-Alpaca” dataset hosted on huggingface.com is used for the model fine-tuning. It is an Alpaca-formatted dataset, which contains “instruction”, “input” and “output” fields. Instruction is a problem statement to write the code in Python programming language and output is the actual code of respective instruction. This dataset contains a total of 22,608 records. An actual example of instruction from the dataset is shown below.

```
Count the number of vowels in the string and return a dictionary where the keys are the vowels and the values are the counts of each vowel. However, you are not allowed to use any built-in functions or libraries that directly count the number of vowels in the string. text = "Hello World!" For example, for the given text, the output should be: {'a': 0, 'e': 1, 'i': 0, 'o': 2, 'u': 0}
```

Figure 4: Sample instruction from the dataset

As stated above the instruction field contains the problem statement to write the Python language code. Along with the problem statement some of the instructions also contain input and output examples, expected time and space complexity of the algorithm.

```
Here is one way to solve the problem without using any built-in functions or libraries that directly count the number of vowels in the string: ```python def count_vowels(text): vowels = ['a', 'e', 'i', 'o', 'u'] vowel_counts = {vowel: 0 for vowel in vowels} for char in text: if char.lower() in vowels: vowel_counts[char.lower()] += 1 return vowel_counts text = "Hello World!" result = count_vowels(text) print(result) ``` Output: {'a': 0, 'e': 1, 'i': 0, 'o': 2, 'u': 0} Explanation: - We define a list 'vowels' containing all the vowels. - We initialize a dictionary 'vowel_counts' with keys as vowels and values as 0. - We iterate over each character in the input string 'text'. - If the lowercase version of the character is in the 'vowels' list, we increment the corresponding value in the 'vowel_counts' dictionary. - Finally, we return the 'vowel_counts' dictionary.
```

Figure 5: Sample output from the dataset

The output field contains the actual Python language code for the problem statement along with its brief explanation. The input field of the dataset is empty and does not contain anything.

4.3 Exploratory Data Analysis

This section concentrates on the EDA of “Vezora/Tested-22k-Python-Alpaca” dataset used for model fine-tuning.

- **Instruction:** The smallest instruction in the dataset is 22 characters long and the longest one is 4754 characters long. The average instruction length is 450 characters long.

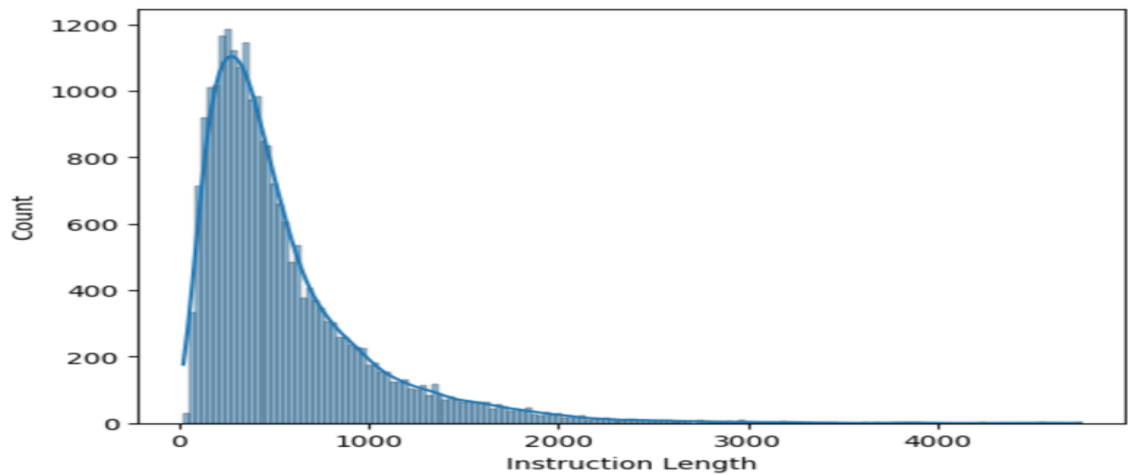


Figure 6: Histogram displaying Instruction length

The length distribution of the instruction field of the dataset is displayed in the above histogram. Most of the instruction has less than 2000 characters and very few are more than 4000 characters long.

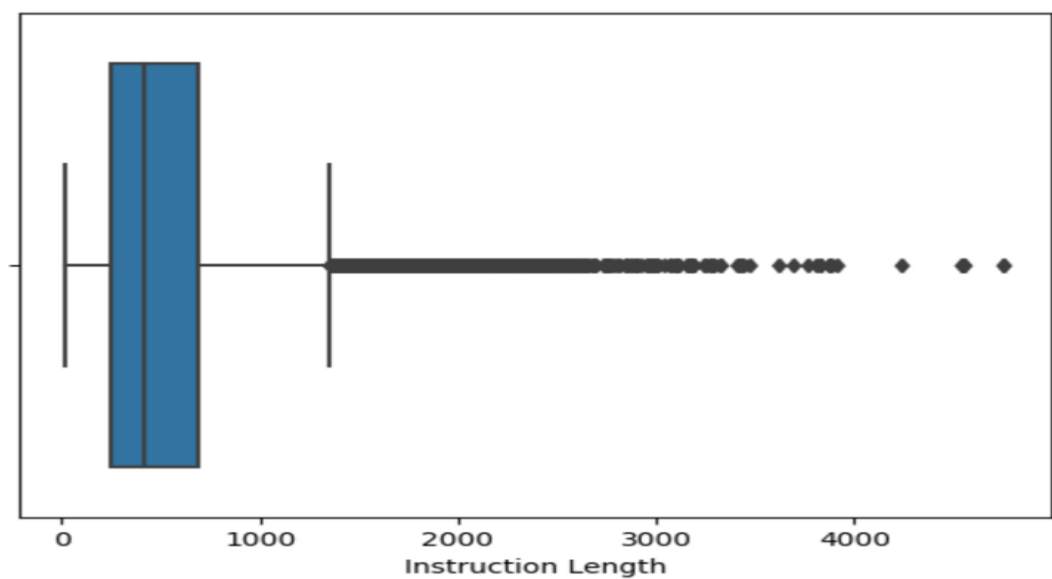


Figure 7: Box plot of the Instruction length

As per the above box plot, more than 75% of the instruction fields are less than 1000 characters long.

- **Output:** The smallest output in the dataset is 48 characters long and the longest one is 11353 characters long. The average output length is 1506 characters long.

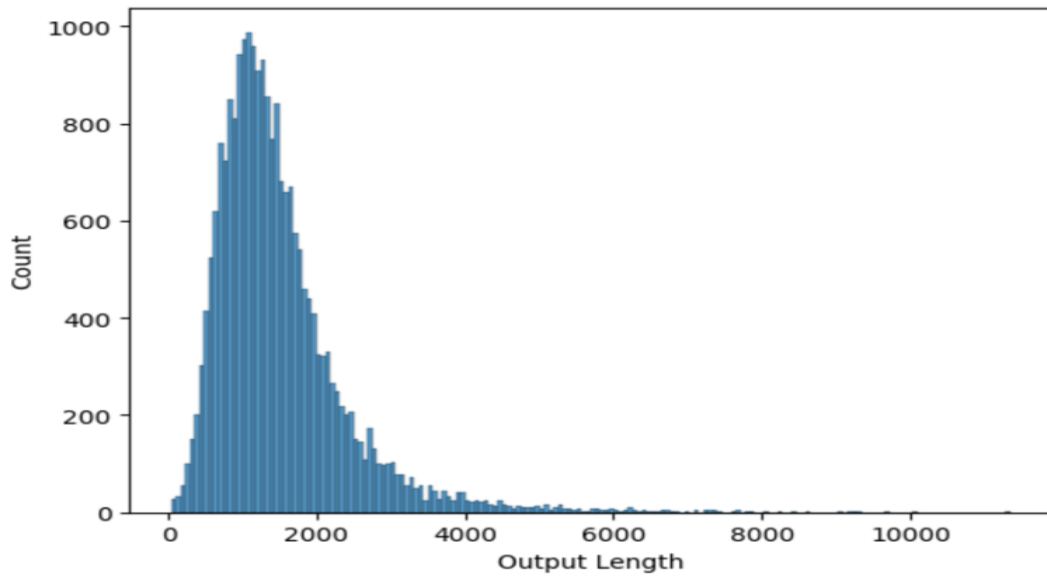


Figure 8: Histogram displaying Output length

The length distribution of the output field of the dataset is displayed in the above histogram. Most of the output field has less than 4000 characters and very few are more than 6000 characters long.

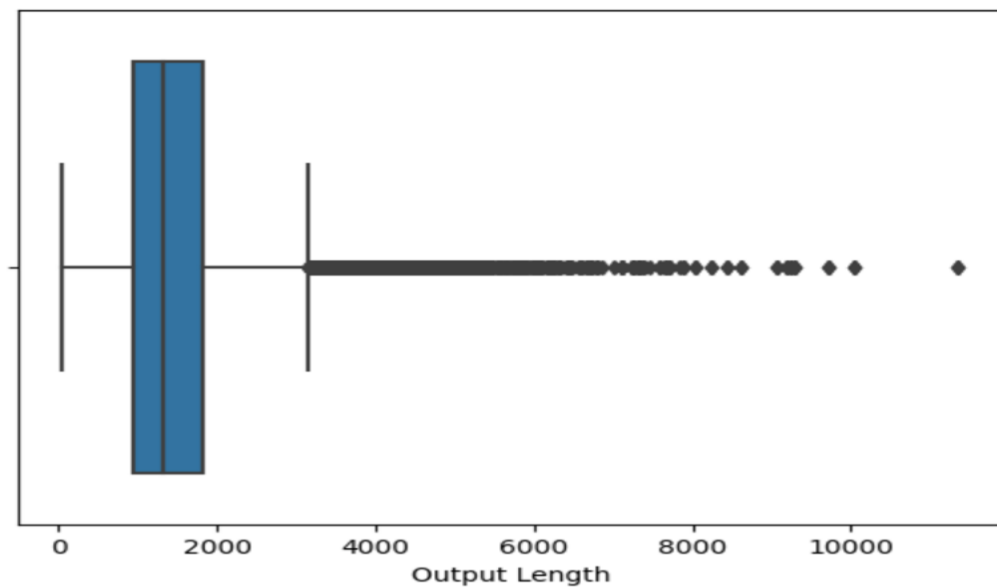
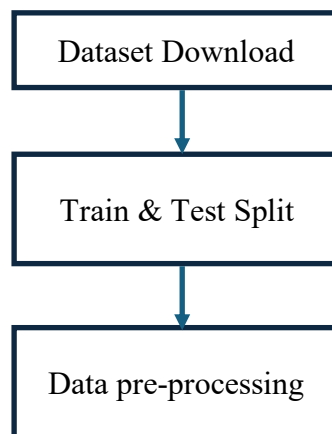


Figure 9: Box plot of the Output length

As per the above box plot, more than 75% of the Output fields are less than 2000 characters long.

4.4 Data Preparation

The steps required for model fine-tuning using the instruction dataset are listed below.



- **Dataset Download:** The dataset required for model fine-tuning will be downloaded from huggingface.com using the huggingface datasets library.
- **Train & Test Split:** The downloaded dataset will be split into train and test datasets, to be used for model training and evaluation respectively. Around 80% of the total records will be used for model training and the remaining 20% will be used for model evaluation.
- **Data Preprocessing:** Records in the dataset have three fields; Instruction, Input, and Output. These data records cannot be used for the model fine-tuning directly. It needs to be converted into the following string format before using it for model fine-tuning.

```
Below is an instruction that describes a task, paired with an input that provides further context.  
Write a response that appropriately completes the request.
```

```
### Instruction:  
{instruction}
```

```
### Input:  
{input}
```

```
### Output:  
{output}
```

Figure 10: Instruction Fine-Tuning prompt format

Placeholders in the above template will be replaced by actual Instruction, Input, and Output from the dataset to convert the JSON data record into text format. This textual format input will be used for instruction fine-tuning of the LLM.

4.5 Hyperparameters

Hyperparameters are a set of external configuration parameters that are used to configure the machine learning model training behaviour. These parameters are not learned during training but specified by the Data Scientist or AI Engineer before the start of model training. Changing the hyperparameters can significantly change the accuracy and performance of the model. Thus selecting the optimum value for hyperparameters is very important.

For LLM fine-tuning using LoRA; both model Training and LoRA configuration parameters are used.

4.5.1 Training Parameters:

TrainingArgument class of the huggingface transformer library provides various parameters to customize for the model training. A few of the important hyperparameters used for model training are specified below.

#	Parameter	Description
1	output_dir	Model checkpoints, predictions & logs are saved in this directory
2	num_train_epochs	Number of training epochs. Default: 3
3	per_device_train_batch_size	Batch size per CPU/GPU for training. Default: 8
4	per_device_eval_batch_size	Batch size per CPU/GPU for evaluation. Default: 8
5	gradient_accumulation_steps	Number of updates steps to accumulate the gradients for, before performing a backward/update pass. Default: 1
6	optim	Optimizer for training. Default: adamw_torch
7	save_steps	Number of updates steps before two checkpoint saves if save_strategy="steps".

8	logging_steps	Number of update steps between two logs if logging_strategy="steps".
9	learning_rate	learning rate for AdamW optimizer.
10	eval_strategy	Training evaluation strategy
11	weight_decay	The weight decay to apply (if not zero) to all layers except all bias and LayerNorm weights in AdamW optimizer.
12	fp_16	If set to true then 16-bit precision training will be used instead of 32-bit
13	bf_16	If set to true then 16-bit precision training will be used instead of 32-bit
14	max_grad_norm	Maximum gradient norm
15	max_steps	If set to a positive number, the total number of training steps to perform. Overrides num_train_epochs.
16	warmup_ratio	Ratio of total training steps used for a linear warmup from 0 to learning_rate.
17	group_by_length	Whether or not to group together samples of roughly the same length in the training dataset
18	lr_scheduler_type	Scheduler type to use for training. Default: linear

4.5.2 LoRA Configuration:

LoRA Configuration is used to configure the Low-Rank Adapters setting required for the Parameter-Efficient Fine-Tuning method. LoraConfig class of the PEFT library from huggingface is used to set the LoRA configuration in Python. Some of the LoRAConfig parameters are as follows:

- **r:** LoRA attention dimension can be configured by setting this parameter. The value of the parameter is directly proportional to the number of parameters to train during the fine-tuning process. The higher the rank more the parameters to fine-tune.
- **lora_alpha:** Parameter for LoRA scaling.
- **target_modules:** LoRA adapters will be applied to modules mentioned in this parameter.

- **lora_dropouts:** LoRA layers dropout probability. This parameter is used to avoid the model overfitting on the training data.
- **bias:** LoRA bias type. Examples are 'none', 'all', or 'lora_only'.
- **task_type:** Task supported by PEFT. Example SEQ_CLS, SEQ_2_SEQ_LM, CAUSAL_LM, TOKEN_CLS, QUESTION_ANS, FEATURE_EXTRACTION

4.6 LLM Fine-Tuning

Large Language Models will be fine-tuned using Full Dataset and Split Dataset Fine-Tuning approaches to understand the effect of dataset size on the model fine-tuning performance in terms of resource utilization, time taken, and inference accuracy.

4.6.1 Full Dataset Fine-Tuning (FDFT):

In this approach, all the records in the dataset will be used for model fine-tuning at once. This way model will be fine-tuned only once using the whole dataset.

tiiuae/falcon-rw-1b, TinyLlama/TinyLlama_v1.1, openai-community/gpt2 model will be fine-tuned with Vezora/Tested-22k-Python-Alpaca dataset. All the records in the dataset will be used for the model fine-tuning process and only one final fine-tuned model will be created at the end of the fine-tuning process. All these models and datasets are hosted on huggingface. Inferences will be generated before and after fine-tuning to compare the accuracy of generated inferences. Model fine-tuning will be performed using both LoRA and QLoRA approaches. For QLoRA model will be downloaded in 4-bit format by setting BitsAndBytesConfig.

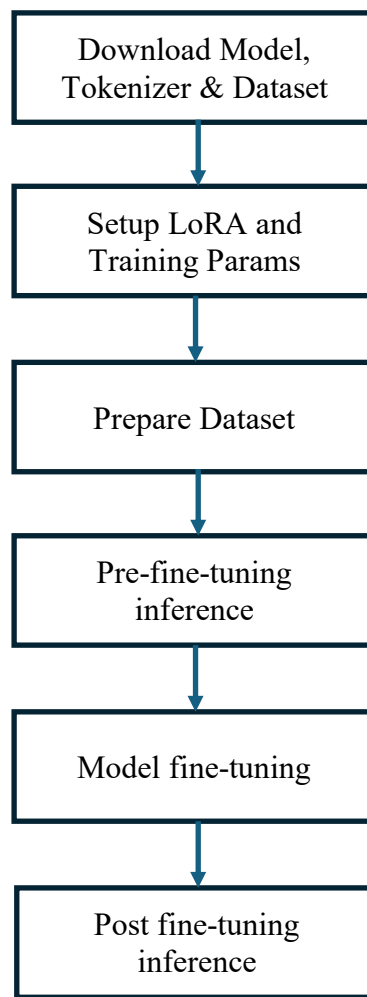


Figure 11: Full Data Fine-tuning Flow Chart

- **Download Model, Tokenizer & Dataset:** All these artifacts will be downloaded from huggingface.com using the huggingface Python libraries. Huggingface has open-sourced multiple libraries to download and use the models, tokenizers, and datasets.

```
# Load model directly
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("meta-llama/Meta-Llama-3.1-8B")
model = AutoModelForCausalLM.from_pretrained("meta-llama/Meta-Llama-3.1-8B")
```

Figure 12: Code snippet to download model & tokenizer from huggingface

- **Model:** LLM downloaded from huggingface is used for fine-tuning. Huggingface is an AI model repository, it hosts different models, datasets, and tokenizers. Huggingface has also open-sourced various libraries and APIs used for model training, inference, and deployment. Huggingface is like the GitHub repository for Artificial Intelligence models and datasets.
- **Tokenizer:** A Tokenizer is used to process the input sent to the model, it breaks the input data into machine-understandable, manageable, and smaller units called tokens. The tokenizer pre-processes the input prompts before passing them to the model for inference. Huggingface hosts various models along with their respective tokenizers. Inference output from model is also passed to the tokenizer to convert it back to human readable format. The huggingface transformer library has methods and classes to download and use both models and tokenizers.
- **Dataset:** Along with models and respective tokenizers huggingface also hosts various datasets required for model training and fine-tuning purpose. Using the huggingface open-source libraries datasets can be downloaded and used for model training and fine-tuning purposes.
- **Setup LoRA & Training Parameters:** Hyperparameters mentioned in Section 4.5 are set before the start of model fine-tuning. Hyperparameters control the overall fine-tuning process behavior.
 - **LoRA Parameter:** LoRA parameters control the rank of the adapters to be trained during the Parameter-Efficient Fine-Tuning. Higher the rank more the parameters to train during fine-tuning and vice versa. Higher rank results in more trainable parameters and higher inference accuracy on the other hand low rank results in a smaller number of trainable parameters and less inference accuracy. So, it's very important to strike the balance between accuracy and resource consumption while deciding the rank of LoRA adapters.
 - **Training Parameters:** Training parameters control the overall fine-tuning process. Output directory to store the model checkpoints, training, and evaluation batch size, learning rate, and evaluation strategy are set. These parameters can be changed as per the expected training outcome.
- **Prepare Dataset:** The dataset downloaded from the huggingface cannot be used as it is, so it needs to be preprocessed before passing it to the model for fine-tuning. The

instruction dataset is in JSON format and has the Instruction, Input, and Output fields. Every JSON record needs to be converted into string format so that it can be used for model fine-tuning. Data will be split into separate training and validation sets for model training and evaluation purpose.

- **Pre-Fine-tuning Inference:** Prompts are sent to the model and inferences are generated. The input prompt is tokenized before it is passed to the model for inference. Generated inferences are recorded for comparison purpose.

```
# Use a pipeline as a high-level helper
from transformers import pipeline

messages = [
    {"role": "user", "content": "Who are you?"},
]
pipe = pipeline("text-generation", model="meta-llama/Meta-Llama-3.1-8B-Instruct")
pipe(messages)
```

Figure 13: Llama-3 Inference code snippet

- **Model Fine-Tuning:** After the data preprocessing, hyperparameter setting, and pre-fine-tuning inference finally model fine-tuning process is started. Parameter-Efficient Fine-Tuning method is used for implementing the model fine-tuning.
 - **LoRA:** This method significantly reduces the number of trainable model parameters by introducing the low-rank adapters. As the number of parameters are reduced, time and computation resources required for fine-tuning are also reduced. This method is very efficient in fine-tuning the model with limited resource and time availability.
 - **QLoRA:** QLoRA is an extension of the LoRA method. Pre-trained model parameters are quantized to 4-bits to save the memory footprints of the model. With QLoRA 65B parameter model can be fine-tuned using a single 48GB CPU without affecting the model performance.
- **Post Fine-Tuning Inference:** After the model fine-tuning process is completed, once again the inferences are generated for the same prompt used in post-fine-tuning inference stage. Both pre and post fine-tuning inferences are compared to analyze the inference accuracy.

4.6.2 Split Dataset Fine-Tuning (SDFT):

In this approach, the single dataset is split into multiple equal sub-datasets. The model is fine-tuned iteratively with the individual subset of the dataset and after each fine-tuning process inferences are generated to analyze the model accuracy. Most of the process in this approach is the same as the Full Dataset Fine-Tuning, except for the part where the model is fine-tuned multiple times with a smaller subset of the dataset.

Similar to Full Dataset Fine-Tuning, tiuae/falcon-rw-1b, TinyLlama/TinyLlama_v1.1, openai-community/gpt2 model will be fine-tuned with Vezora/Trained-22k-Python-Alpaca dataset. This dataset has a total of 22608 records, these records will be divided into 4 separate datasets.

Datasets 1, 2, and 3 will have 5500 records, and dataset 4 will have the remaining 6108. After fine-tuning the model with dataset 1, an intermediate fine-tuned model will be generated. This model will be further fine-tuned with dataset 2. This process will be continued till the model is fine-tuned with all the remaining datasets. Inferences will be generated for all the intermediate and final fine-tuned models. The detailed flow chart explaining the process of Split Data Fine-Tuning process is as below.

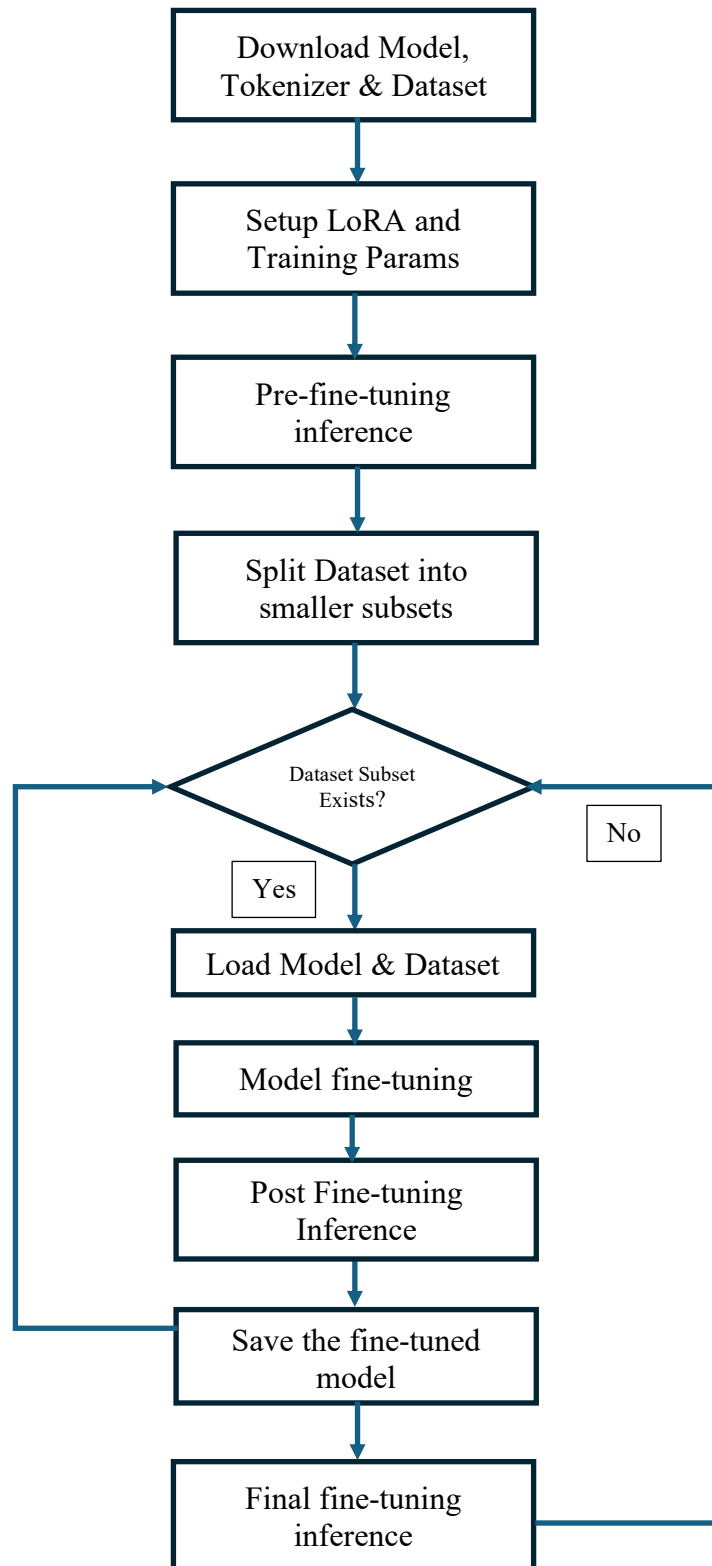


Figure 14: Split Data Fine-tuning Flow Chart

- **Download Model Dataset & Tokenizer:** This part is same as the Full Dataset Fine-Tuning approach. The model, respective tokenizer, and the dataset will be downloaded from the huggingface repository using huggingface open-source libraries.
- **Setup LoRA & Training Parameters:** Hyperparameters for both the Full Dataset Fine-Tuning approach and Split Dataset Fine-Tuning approach are the same.
- **Pre-Fine-Tuning Inference:** Prompts are sent to the raw model and inferences are generated. Generated inferences are recorded for comparison purposes.
- **Split Dataset into multiple equal subsets:** In this step, the original dataset downloaded from huggingface and split into multiple equal datasets. The model will be trained iteratively with each dataset independently.

e.g. Dataset having 20,000 records can be split into 4 separate datasets having 5000 records each.

Dataset used for fine-tuning.

- Vezora/Tested-22k-Python-Alpaca

- **Load Model and Dataset:** For the first time fine-tuning the dataset and model will be loaded from the huggingface repository. Once the fine-tuning process is completed model will be saved on the local disk. For subsequent fine-tuning processes, for all the remaining subset of datasets, the model is loaded on a local disk and will be fine-tuned. After completion of the fine-tuning process model will be saved back to the local disk. This process will be repeated for all remaining subsets of datasets. The transformer library will download the model from huggingface if the correct model name is specified. The model will be downloaded from a local disk if a local directory path is provided as the model name.

```
# Load model from local directory
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained("/home/ec2-user/SageMaker/gpt-2/LoRA")
```

Figure 15: Code snippet to load model from local disk

- **Model Fine-Tuning:** The model will be fine-tuned using the multiple smaller subsets of the datasets. After the model fine-tuning process is completed, the model will be stored on a local disk for further fine-tuning by the remaining datasets.

The following models are fine-tuned for the experiment purpose.

- openai-community/gpt2
 - tiuae/falcon-rw-1b
 - TinyLlama/TinyLlama_v1.1
- **Post Fine-Tuning Inference:** After each fine-tuning process inference will be generated using the fine-tuned model.
 - **Save Fine-Tuned Model:** At the end of fine-tuning, a model will be saved for subsequent fine-tuning process using the remaining datasets.
 - **Final Fine-Tuning Inference:** After completion of all the fine-tuning processes, inference will be generated on the final fine-tuned model. These inferences will be used to compare the accuracy of Full Dataset Fine-Tuning and Split Dataset Fine-Tuning approaches.

4.7 Model Inference

Zero-Shot prompting technique is used to generate the inference from models. In the Zero-Shot prompting technique other than the input prompt no other information such as examples or demonstrations are provided to the model for generating the inference. The model generates the inference only based on the instructions given in the input prompt.

Can you design a program in Python that can predict the likelihood of a certain material becoming unstable under high pressure and temperature conditions? This can be calculated using the Gurney equation, which takes into account parameters such as the chemical composition and crystal structure of the material. Additionally, can you provide a real-world scenario where this program could be applied in the field of material science and engineering?

Figure 16: Zero-Shot prompt example

In the above Zero-Shot prompt example other than the instruction to the model no other additional information such as examples or references are provided to the model. Few-Shot prompting and Chain-of-Thoughts are other widely used prompting techniques. In these

techniques along with the input prompt some supporting examples or additional information is also provided to the model for reference.

4.8 GPU & Memory Metric Collection

GPU & memory utilization metric is collected using the NVIDIA System Management Interface Utility (nvidia-smi). It is a command-line utility that provides the management and monitoring capabilities for various NVIDIA GPUs. Memory and GPU utilization, the temperature of the GPU, and other details are collected every 10 seconds using nvidia-smi command. The output of this command is saved in CSV format for further analysis.

```
nvidia-smi -l 10 --query-gpu=utilization.gpu,utilization.memory,temperature.gpu,memory.total,memory.used,memory.free \
--format=csv,nounits -f /home/ec2-user/SageMaker/tinyllama/split-dataset-logs-v1.csv
```

Figure 17: nvidia-smi command to collect the GPU metric every 10 seconds

4.9 AWS SageMaker

AWS SageMaker Jupyter Notebook instances are used to run the model fine-tuning process. AWS SageMaker provides different types of GPU-enabled EC2 instances on demand basis to be used for various purposes. GPU is a special type of hardware essential for deep learning Artificial Intelligence model training and serving.

AWS ml.g5.2xlarge EC2 instance is used for the model fine-tuning. This instance comes with 1 NVIDIA A10G GPU and 24 GB memory. This much GPU and memory is enough to train or fine-tune the Small Language Models (SML) which contain around 1 billion parameters. Higher configuration GPUs are required to train or fine-tune the larger models containing tens of billions of parameters or more. AWS SageMaker is a pay-per-use service, and the EC2 ml.g5.2xlarge instance costs USD 1.212 per hour for on-demand basic.

4.10 Summary

All the implementation details about Full Dataset Fine-Tuning and Split Dataset Fine-Tuning are discussed in this chapter. Details about the training and LoRA hyperparameters are also discussed in details. Huggingface hosts the models, tokenizers, and datasets and has also published various libraries useful for model training, fine-tuning, and other operations. All results of the fine-tuning operations will be discussed in the next chapter.

CHAPTER 5: RESULTS AND EVALUATIONS

5.1 Introduction

Results and analysis of the experiments performed in Chapter 4 are discussed in this chapter. The time and resources required for fine-tuning the LLM along with the accuracy of the LLM are explained in the following sections. The results and evaluation of Full Data Fine-Tuning method and Split Data Fine-Tuning method will help the Data Scientist and AI Engineers to decide the frequency and size of the dataset to be used for fine-tuning the Large Language Models.

- **Resource Requirements:** GPU and memory utilized to fine-tune the LLM using the Full Dataset Fine-Tuning method and Split Dataset Fine-Tuning method are explained for both LoRA and QLoRA techniques.
- **Time Required:** The time required to fine-tune the LLM using the Full Dataset Fine-Tuning method and Split Dataset Fine-Tuning method is explained for both LoRA and QLoRA techniques.
- **Accuracy of the LLM Inference:** The accuracy of the model after fine-tuning using the Full Dataset Fine-Tuning approach and Split Dataset Fine-Tuning approach is also explained for both LoRA and QLoRA techniques.

5.2 Resource Requirements

LLM fine-tuning needs a specialized high-performance GPU to run the fine-tuning jobs. Because of immense demand, sometimes these GPUs are hard to get and are very expensive to buy or rent. Comparing the resources required for fine-tuning is important so that Data Scientists and AI Engineers can use the most cost-effective method to fine-tune the LLMs. GPU and Memory required for Full Dataset Fine-Tuning and Split Dataset Fine-Tuning are shown in the below-mentioned tables. AWS EC2 ml.g5.2xlarge instance has one GPU and 24 GiB of GPU memory it is enough to fine-tuned or serve the model having around 1 billion parameters.

- **Low Rank Adapters:**

Memory and GPU resources required for LoRA fine-tuning for both Full Dataset Fine-Tuning and Split Dataset Fine-Tuning approach are listed in below mentioned table.

Table 1: LoRA fine-tuning resource utilization in percentage

LLM	Metric	Full Dataset	Split Dataset				
			Dataset 1	Dataset 2	Dataset 3	Dataset 4	Average
TinyLlama	GPU	97.37	94.58	96.35	95.96	96.12	95.75
	Memory	86.18	84.19	85.73	86.06	86.94	85.73
GPT-2	GPU	83.74	69.76	77.08	79.74	65.08	72.91
	Memory	71.3	59.97	65.08	68.28	65.08	65.94
Falcon-RW-1B	GPU	95.8	89.9	93.71	92.98	93.84	92.61
	Memory	72.68	68.17	71.45	70.74	71.68	70.51

Note: All the values are in percentage.

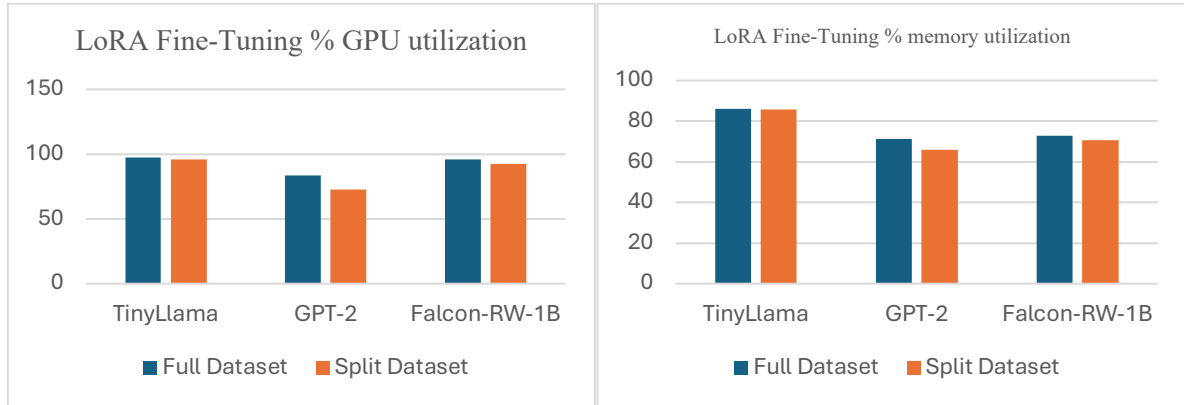


Figure 18: LoRA fine-tuning resource requirement

Some of the key observations from the above table and graph of the LoRA fine-tuning method about the memory and GPU utilization are mentioned below:

- The SDFT approach consumes less compute and memory compared to FDFT.
- The SDFT approach consumes around 2% less GPU and 1% less memory compared to FDFT for fine-tuning the TinyLlama models.
- The SDFT approach consumes around 13% less GPU and 9% less memory compared to FDFT for fine-tuning the GPT-2 models.

- The SDFT approach consumes around 3% less GPU and memory compared to FDFT for fine-tuning the Falcon-RW-1B models.
- GPU and Memory savings are more for fine-tuning the GPT-2 models and less for the TinyLlama model.
- Overall, from the above results it can be concluded that the SDFT approach consumes less GPU and memory for fine-tuning the models using LoRA technique compared to the FDFT approach for the same volume of data.

- **Quantized Low-Rank Adapters (QLoRA):**

Resources required for QLoRA fine-tuning for both Full Dataset Fine-Tuning and Split Dataset Fine-Tuning approaches are listed below mentioned table.

Table 2: QLoRA fine-tuning resource utilization in percentage

LLM	Metric	Full Dataset	Split Dataset				
			Dataset 1	Dataset 2	Dataset 3	Dataset 4	Average
TinyLlama	GPU	94.2	96.2	95.96	95.76	95.41	95.83
	Memory	86.89	83.17	82.84	86.06	82.90	83.83
GPT-2	GPU	85.58	82.82	78.7	79.41	83.66	81.15
	Memory	70.81	69.67	66.42	65.35	69.01	67.61
Falcon-RW-1B	GPU	89.39	95.43	95.43	93.56	96.36	92.61
	Memory	70.20	76.85	76.85	75.95	78.95	70.51

Note: All the values are in percentage

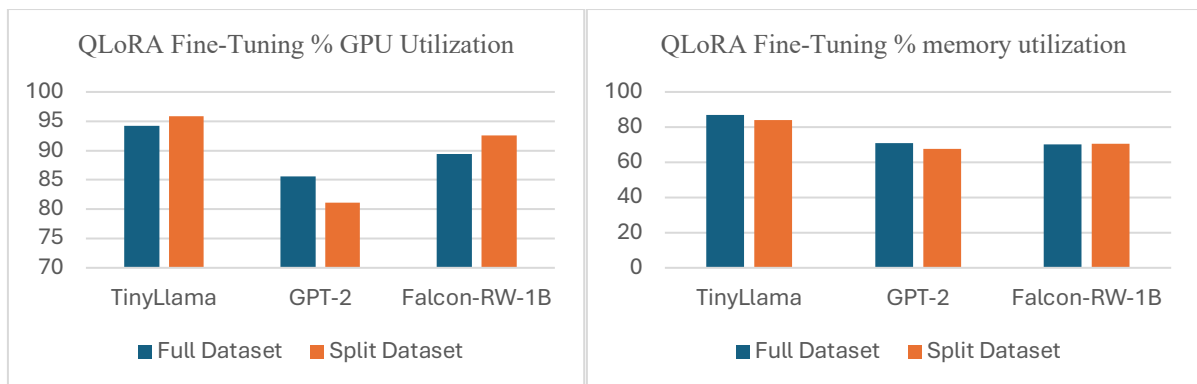


Figure 19: QLoRA fine-tuning resource requirement

Some of the key observations about the QLoRA fine-tuning method regarding the memory and GPU consumption are mentioned below:

- The SDFT approach utilizes more GPU for fine-tuning the TinyLlama and Falcon-RW-1B models on the other hand GPU utilization is less for fine-tuning the GPT-2 models.
- The SDFT approach consumes around 1% more GPU and 1% less memory compared to FDFT for fine-tuning the TinyLlama model. So, there is no clear advantage of any approach over other in case of TinyLlama model fine-tuning.
- The SDFT approach consumes around 4% less GPU and memory compared to FDFT for fine-tuning the GPT-2 models.
- The FDFT approach consumes around 5% less GPU and approximately same memory for fine-tuning Falcon-RW-1B models
- Overall, from the above results it can be concluded that the FDFT approach consumes less GPU and memory for fine-tuning the models using the QLoRA method compared to the SDFT approach for the same amount of data.

5.3 Time Required for Fine-Tuning

The time required for LLM fine-tuning directly translates to the cost required. It could take couple of weeks and hundreds of GPUs to train the larger models. Even a small improvement in the time taken by LLM for fine-tuning could translate into bigger cost savings. So it is also very important to check the time required for the fine-tuning. This analysis will help to choose the fastest way to fine-tune the LLM without affecting the performance and accuracy.

The time required for Full Dataset Fine-Tuning and Split Dataset Fine-Tuning is mentioned in the following table.

- **Low Rank Adapters:**

The time required for LoRA fine-tuning for both Full Dataset Fine-Tuning and Split-Dataset Fine-Tuning is listed below mentioned table and graph.

Table 3: LoRA fine-tuning time taken

Model	Full Dataset	Split				
		Dataset 1	Dataset 2	Dataset 3	Dataset 4	Total
TinyLlama	02:09:24	00:34:56	00:24:04	00:30:22	00:38:20	02:07:52
GPT-2	00:31:25	00:08:54	00:05:14	00:07:05	00:09:24	00:30:37
Falcon-RW-1B	01:26:02	00:23:28	00:16:21	00:20:03	00:24:22	01:24:14

Note: Time is in hh:mm:ss format.

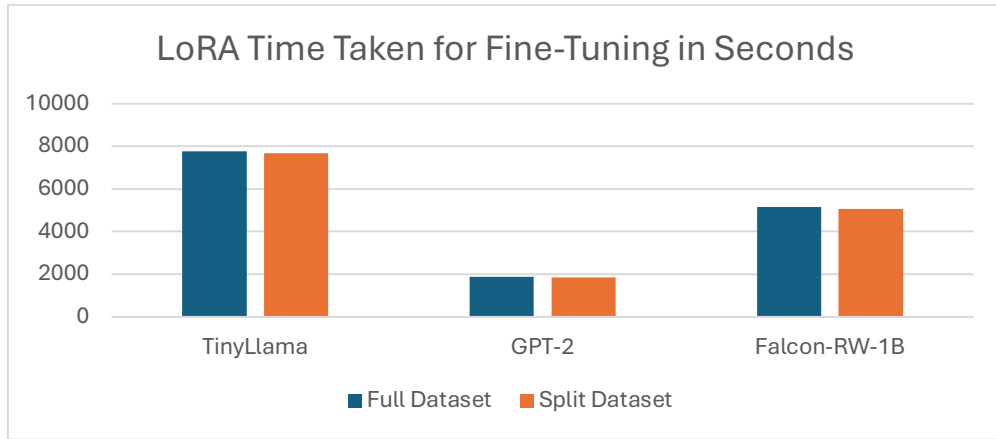


Figure 20: LoRA fine-tuning time taken

Some of the key observations about the time required for the LoRA fine-tuning method are mentioned below:

- The SDFT model fine-tuning approach is faster than the FDTF approach by 2-3%.
- So basically, the SDFT method takes marginally less time for fine-tuning than the FDTF method for the same data size.

- **Quantized Low-Rank Adapters (QLoRA):**

Time required for QLoRA fine-tuning for both Full Dataset Fine-tuning and Split-Dataset Fine-tuning is listed below mentioned table.

Table 4: QLoRA fine-tuning time taken

Model	Full Dataset	Split				
		Dataset 1	Dataset 2	Dataset 3	Dataset 4	Total
TinyLlama	02:07:27	00:36:10	00:24:41	00:30:57	00:39:35	02:10:23
GPT-2	00:27:36	00:06:55	00:04:33	00:05:52	00:07:22	00:24:42
Falcon-RW-1B	01:35:06	00:24:39	00:15:05	00:19:59	00:25:21	01:25:04

Note: Time is in hh:mm:ss format.

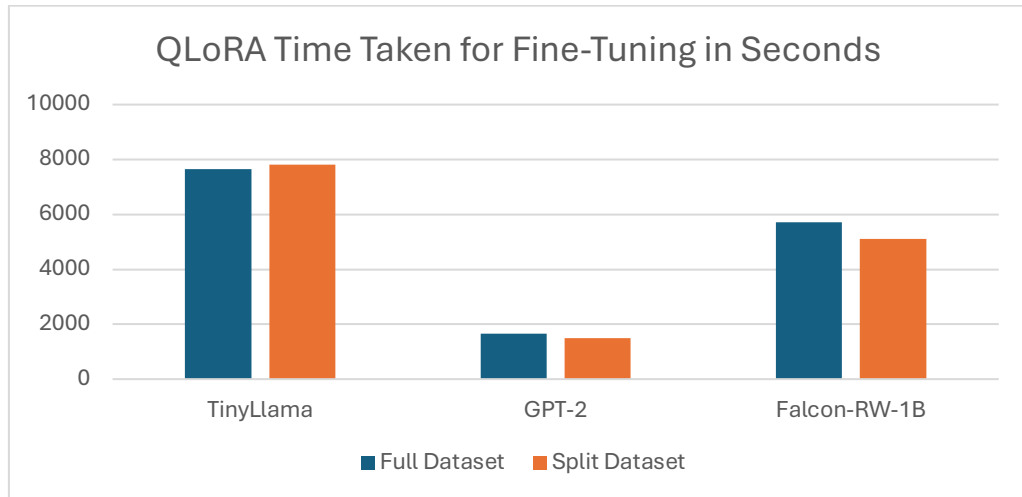


Figure 21: QLoRA fine-tuning time taken

Some of the key observations about the time required for the QLoRA fine-tuning method are mentioned below:

- Unlike the LoRA fine-tuning approach, there is no clear winner for the QLoRA method.
- GPT-2 and Falcon-1B-RW take 10% less time for QLoRA fine-tuning using the SDFT approach compared to the FDTF approach.
- On the other hand, TinyLlama takes 2% more time for QLoRA fine-tuning using the SDFT approach compared to the FDTF approach.

5.4 Accuracy of the LLM Inference

LLM inference should generate accurate facts and relevant answers to build the confidence of users and increase public or production adoption. Users will be deterred if LLM inference generates wrong facts and irrelevant answers. Alphabet's (Google's parent company) shares slid 9% and lost around USD 100 billion in market capitalization when Google's new AI ChatBot Bard, generated factually incorrect answers in a promotional video posted on X. (formerly known as Twitter)

So, along with resource consumption and time required for fine-tuning the LLM, the accuracy and relevance of the inference generated by LLM are also extremely important. The ability of the model to generate the code for a given fine-tuned model was checked to determine the accuracy. The accuracy or correctness of the generated python code was not validated.

The same prompts were used to generate the inference before and after fine-tuning. Inferences were also generated after fine-tuning the model with each sub-dataset for the Split Dataset Fine-Tuning method.

The prompt contains the problem statement to generate the Python language code. Some of the prompts may also contain other instructions like required time and space complexity, return type and some other corner cases and special conditions to be handled by the Python code.

```
"""Create a function that removes duplicates from an array and returns an array of the unique values. The function should only use O(1) additional space and have a time complexity of O(n), where n is the length of the input array. The input array can contain integers, floating-point numbers, strings, and nested arrays. The output array should be sorted in descending order. Additionally, the function should handle nested arrays correctly by recursively flattening them before removing duplicates."""
```

Figure 22: Prompt to generate Python code with expected time and space complexity

```
"""Create a function that takes two parameters, a number and a string, and returns an array with those two values in it. The string parameter should be checked if it is a valid string and if it contains only alphabetical characters. If the string is not valid (length less than 10 characters) or contains non-alphabetical characters, the function should return an empty array. Additionally, the number parameter should be restricted to a range between -1000 and 1000 (inclusive). The function should also check if the number is a prime number and return an empty array if it is not."""
```

Figure 23: Prompt to generate Python code without special instructions

- **Full Dataset Fine-Tuned Model Inference with generated Python Code**

```
***** PROMPT *****
Create a 3x4 NumPy array of random integers from 0 to 5, where each row should have at least one unique value.
***** Inference *****
Create a 3x4 NumPy array of random integers from 0 to 5, where each row should have at least one unique value. At l
east one random integer at an offset should always go to a valid random integer, then any other integers inbetween
should be appended to the original pair. Finally the final two rows should follow the same pattern as the first.

###python

Let's implement a float that iterates until the last row is in the array and keeps using the same array_len string
to iterate through the elements in the array.

###python

###python
elsepython

###python

###python

Then in Python
print("The number of items in the array: 10")
# prints "10"
4    print("The number of remaining rows in the array: 10)
      print("The number of numbers in the array: 10)
      # prints "6"
      # prints "5"
      #

###python
#
#
#
def calculate_array_len(arr):
def sum(num):
print
```

Figure 24: GPT-2 Full Dataset fine-tuning inference with generated Python code

- **Full Dataset Fine-Tuned Model Inference without Python Code**

```
***** PROMPT *****
Find the index of the element 'c' in the following list, but the list may contain duplicates and the element 'c' ma
y appear multiple times.
***** Inference *****
Find the index of the element 'c' in the following list, but the list may contain duplicates and the element 'c' ma
y appear multiple times.

The second element of an element must contain at least three consecutive duplicates of the preceding one. In the ca
ses where multiple duplicates of the same element are present in the list, the list will keep updating until it enc
ounters an additional duplicate element.

For example, instead of a list containing four consecutive duplicates of the same element (e.g., ['a', 'b', 'c']),
a list contains only two consecutive duplicates of the same element. If only one duplicates of the same element are
present in the list, it remains unchanged. However, the list containing multiple consecutive duplicates of the same
element must contain at least three consecutive duplicates of the same element. The list may contain duplicates, an
d each duplicate of the same element must contain at least four consecutive duplicates of an instance variable.

Note: If any duplicates of the same element are present in the list, the list may contain duplicate types. This lis
t may contain any type of any duplicate type that is encountered in the given case.

### Response Response to `add_array_of
```

Figure 25: GPT-2 Full Dataset fine-tuning inference without Python code

- **Low Rank Adapters (LoRA):**

Statistics of pre- and post-fine-tuning code generation capabilities of LLM are mentioned in the following table. LoRA method was used, and models were fine-tuned using both the Full Dataset Fine-Tuning and Split Dataset Fine-Tuning approach.

Table 5: LoRA fine-tuning Inference Accuracy

Model	Before Fine-Tuning	Full Dataset	Split				
			Dataset 1	Dataset 2	Dataset 3	Dataset 4	Final
TinyLlama	0	0	5	0	0	0	0
GPT-2	3	6	5	3	1	2	2
Falcon-RW-1B	0	9	4	8	5	5	5

Note: Number of model-generated Python code.

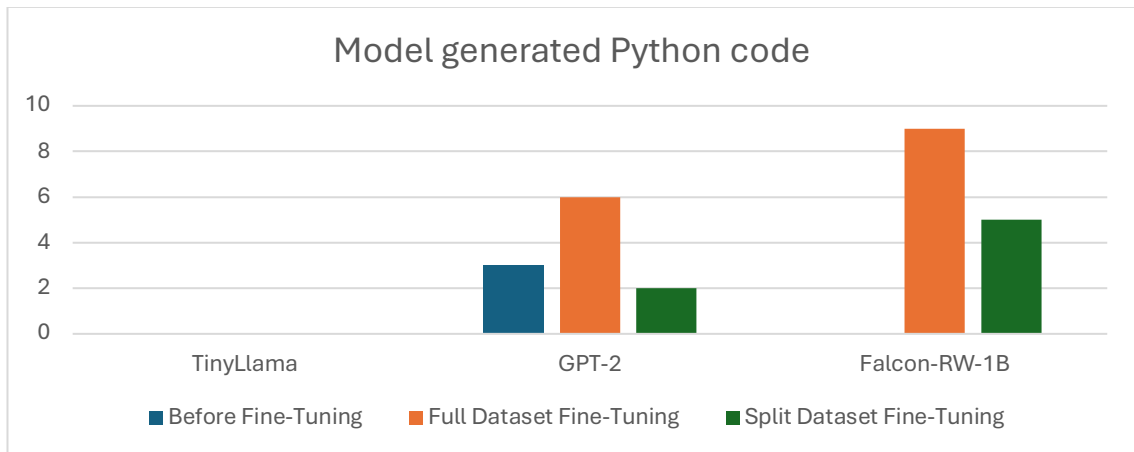


Figure 26: Fine-tuned LoRA model generated Python Code

Some of the key observations about the inference accuracy of the model fine-tuned using the LoRA method are mentioned below:

- The TinyLlama model has no effect of either FDFT or SDFT method on the inference accuracy.
- Both GPT-2 and Falcon-1B-RW showed a decrease in accuracy when fine-tuned using the SDFT method compared to the FDFT method.
- Intermediate models showed better accuracy compared to the final model version for the SDFT approach.

- **Quantized Low-Rank Adapters (QLoRA):**

Statistics of pre- and post-fine-tuning code generation capabilities of LLM are mentioned in the following table. QLoRA technique of PEFT method was used, and models were fine-tuned using both the Full Dataset Fine-Tuning and Split Dataset Fine-Tuning approach.

Table 6: QLoRA fine-tuning Inference accuracy

Model	Before Fine-tuning	Full Dataset	Split				
			Dataset 1	Dataset 2	Dataset 3	Dataset 4	Final
TinyLlama	0	0	4	0	2	0	0
GPT-2	0	3	5	1	2	1	1
Falcon-RW-1B	0	2	1	2	0	7	7

Note: Number of model-generated Python code.

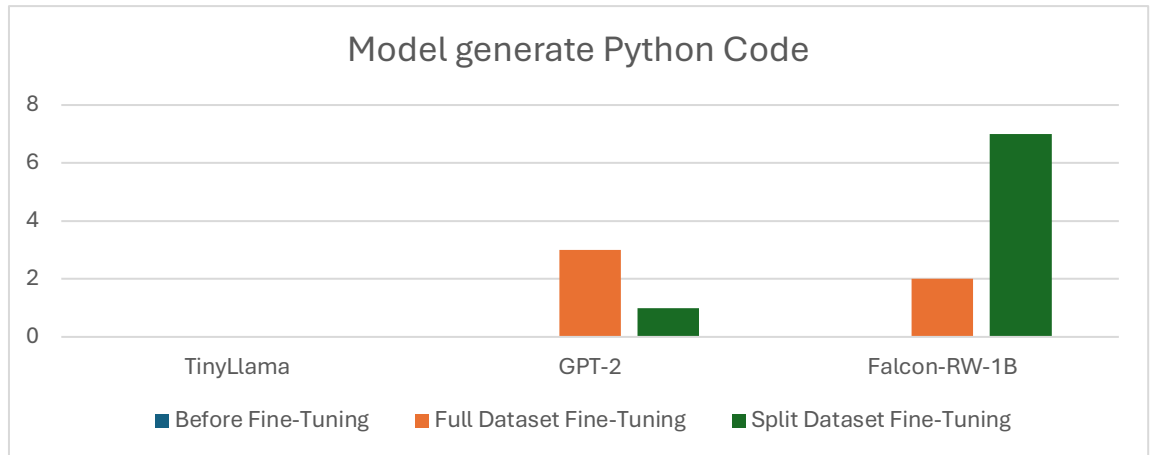


Figure 27: Fine-tuned QLoRA model generated Python Code

Some of the key observations about the inference accuracy of the model fine-tuned using the QLoRA method are mentioned below:

- The TinyLlama model has no effect of either FDFT or SDFT method on the model inference accuracy.
- GPT-2 showed better accuracy for the FDFT method compared to the SDFT method of model fine-tuning.

- Falcon-1B-RW showed better accuracy for the SDFT method compared to the SDFT method for model fine-tuning.
- The LoRA method showed mixed accuracy results for both the SDFT and FDFT approach.
- From the above results, it looks like the accuracy of the model depends on the model and its architecture than the method of fine-tuning.

5.5 Summary

As per the above results SDFT method is faster than the FDFT method for most of the model fine-tuning process. The SDFT method also marginally consumes less GPU and memory for fine-tuning the models than the FDFT method.

Both methods showed mixed results for the accuracy of the inferences generated by the fine-tuned model. It looks like model accuracy is not dependent on the fine-tuning process but rather depends on the model and model architecture. Overall SDFT method of fine-tuning the LLM showed marginally better results than the FDFT method.

CHAPTER 6: CONCLUSION AND RECOMMENDATIONS

6.1 Introduction

Some of the conclusions and recommendations from the above fine-tuning implementations and observations are as follows. Also the future work required for more conclusive report and details are mentioned below.

6.2 Conclusions

6.2.1 Model Fine-Tuning Resource Consumption

In most cases, the Split Dataset Fine-Tuning method consumes less GPU and memory for model fine-tuning compared to Full Dataset Fine-Tuning method for both LoRA and QLoRA approaches of the Parameter Efficient Fine-Tuning technique.

With the emergence of LLM, the demand for GPUs has increased multi-fold. Using the SDFT approach an LLM can be fine-tuned with fewer resources. Thus decreasing the resource requirement and the total cost of model fine-tuning.

Overall, SDFT requires less GPU and memory for model fine-tuning compared to FDFT.

6.2.2 Faster Model Fine-Tuning

As mentioned in the earlier section, 5 out of 6 results showed that the Split Dataset Fine-Tuning method takes marginally less time to fine-tune the LLM compared to the Full Dataset Fine-Tuning method for both the LoRA and QLoRA approach.

QLoRA model fine-tuning with SDFT is 10% faster than QLoRA with FDFT. Bigger LLM takes a few days or weeks to complete the model training or fine-tuning, even a few percentage of time savings could result in significant time and cost savings.

Overall, the SDFT method of model fine-tuning is faster than FDFT.

6.2.3 Inference Accuracy

Inference accuracy has shown opposite results than the time and resource consumption results. For both resource and time metrics Split Dataset Fine-Tuning method turns out to be better than the Full Dataset Fine-Tuning method. But when it comes to model inference accuracy Full Dataset Fine-tuning method is better than the Split Dataset Fine-Tuning. Maybe more research with other models, datasets, and fine-tuning approaches is required before concluding any results about the model inference accuracy.

6.3 Future Work and Recommendations

Future work to have more conclusive and strong arguments for the model fine-tuning approach is described in the following section.

6.3.1 Use Better GPU

Because of cost limitation, only one instance of AWS EC2 ml.g5.2xlarge (1 NVIDIA A10G Tensor Core GPU, 24 GiB Memory) was used for model fine-tuning. This instance costs USD 1.212 per hour for on-demand basis. With better hardware, it will be possible to fine-tune the larger LLMs.

Only Small Language Models (SLM) with around 1 billion parameters were fine-tuned and results were recorded. Using the more powerful hardware, will allow to fine-tune the larger models with tens or hundreds of billions of parameters. Such as falcon-7b-instruct, falcon-180B-chat, Meta-Llama-3-8B, Meta-Llama-3.1-405B.

6.3.2 Use of Larger Datasets

The dataset used for the fine-tuning in this research (vezora/Tested-22k-Python-Alpaca) only contains 22k records. This dataset was used for fine-tuning to control the model fine-tuning cost and time. Fine-tuning the models with larger datasets could give better results about model inference accuracy. LLMs normally perform better when trained with larger datasets having millions of records.

6.3.3 Other Parameter-Efficient Fine-Tuning Methods

In this research, only LoRA and QLoRA PEFT methods were studied for model fine-tuning to understand the resource and time consumption and inference accuracy. Other PEFT methods such as Prefix Tuning, Prompt Tuning, and P-tuning, etc. should be studied to have a more conclusive report on resources and time required for fine-tuning using SDFT and FDFT methods.

6.3.4 Hyperparameter Tuning

Because of time constraints models were fine-tuned for only 3 epoch and without tuning the hyperparameters. Models should be fine-tuned with different set of hyperparameters to understand the effect of epoch, learning rate, batch size, optimizers etc on the resource consumption, time taken and model accuracy.

6.4 Contribution

This research has contributed the following observations to the existing literature on LLM fine-tuning and generative artificial intelligence.

- Comparison between the LLM fine-tuning with larger datasets at larger time intervals versus smaller datasets at smaller time intervals.
- Time and resource consumption for different model fine-tuning approaches.

6.5 Summary

Conclusions and observations based on the results mentioned in Section 5 are discussed in this chapter. Future work and recommendations in order to have more concrete conclusions about the FDFT and SDFT model fine-tuning approaches are also listed in this chapter.

REFERENCES

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention Is All You Need*. <http://arxiv.org/abs/1706.03762>
- Han, Z., Gao, C., Liu, J., Zhang, J., & Zhang, S. Q. (2024). *Parameter-Efficient Fine-tuning for Large Models: A Comprehensive Survey*. <http://arxiv.org/abs/2403.14608>
- Liu, H., Tam, D., Muqeeth, M., Mohta, J., Huang, T., Bansal, M., & Raffel, C. (2022). *Few-Shot Parameter-Efficient Fine-tuning is Better and Cheaper than In-Context Learning*. https://proceedings.neurips.cc/paper_files/paper/2022/hash/0cde695b83bd186c1fd456302888454c-Abstract-Conference.html
- Gao, C., & Zhang, S. Q. (2024). *DLoRA: Distributed Parameter-Efficient Fine-tuning Solution for Large Language Model*. <http://arxiv.org/abs/2404.05182>
- Wu, Z., Arora, A., Wang, Z., Geiger, A., Jurafsky, D., Manning, C. D., & Potts, C. (2024). *ReFT: Representation Finetuning for Language Models*. <http://arxiv.org/abs/2404.03592>
- Arvidsson, S., & Axell, J. (2023). *Prompt engineering guidelines for LLMs in Requirements Engineering*.
- Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs*. https://proceedings.neurips.cc/paper_files/paper/2023/hash/1feb87871436031bdc0f2beaa62a049b-Abstract-Conference.html
- Aghajanyan, A., Zettlemoyer, L., & Gupta, S. (2020). *Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-tuning*. <http://arxiv.org/abs/2012.13255>
- Grosjean, J., & Vamvas, J. (2024). *Fine-tuning the SwissBERT Encoder Model for Embedding Sentences and Documents*. <http://arxiv.org/abs/2405.07513>

- Xu, J., Saravanan, K., van Dalen, R., Mehmood, H., Tuckey, D., & Ozay, M. (2024). *DP-DyLoRA: Fine-tuning Transformer-Based Models On-Device under Differentially Private Federated Learning using Dynamic Low-Rank Adaptation*. <http://arxiv.org/abs/2405.06368>
- Jie, S., Tang, Y., Ding, N., Deng, Z.-H., Han, K., & Wang, Y. (2024). *Memory-Space Visual Prompting for Efficient Vision-Language Fine-tuning*. <http://arxiv.org/abs/2405.05615>
- Gekhman, Z., Yona, G., Aharoni, R., Eyal, M., Feder, A., Reichart, R., & Herzig, J. (2024). *Does Fine-tuning LLMs on New Knowledge Encourage Hallucinations?* <http://arxiv.org/abs/2405.05904>
- Liu, Z., & Luo, J. (2024). *AdaMoLE: Fine-tuning Large Language Models with Adaptive Mixture of Low-Rank Adaptation Experts*. <http://arxiv.org/abs/2405.00361>
- Yang, Y., Liu, X., Gao, T., Xu, X., & Wang, G. (2024). *SA-FedLora: Adaptive Parameter Allocation for Efficient Federated Learning with LoRA Tuning*. <http://arxiv.org/abs/2405.09394>
- Tian, C., Shi, Z., Guo, Z., Li, L., & Xu, C. (2024). *HydraLoRA: An Asymmetric LoRA Architecture for Efficient Fine-tuning*. <http://arxiv.org/abs/2404.19245>
- Wu, X., Huang, S., & Wei, F. (2024). *Mixture of LoRA Experts*. <http://arxiv.org/abs/2404.13628>
- Gao, Z., Wang, Q., Chen, A., Liu, Z., Wu, B., Chen, L., & Li, J. (2024). *Parameter-Efficient Fine-tuning with Discrete Fourier Transform*. <http://arxiv.org/abs/2405.03003>
- Zhao, J., Wang, T., Abid, W., Angus, G., Garg, A., Kinnison, J., Sherstinsky, A., Molino, P., Addair, T., & Rishi, D. (2024). *LoRA Land: 310 Fine-tuned LLMs that Rival GPT-4, A Technical Report*. <http://arxiv.org/abs/2405.00732>
- Haque, S., Eberhart, Z., Bansal, A., & McMillan, C. (2022). Semantic Similarity Metrics for Evaluating Source Code Summarization. *IEEE International Conference on Program Comprehension, 2022-March*, 36–47. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

- Li, D., Ma, Y., Wang, N., Cheng, Z., Duan, L., Zuo, J., Yang, C., & Tang, M. (2024). *MixLoRA: Enhancing Large Language Models Fine-tuning with LoRA based Mixture of Experts*. <http://arxiv.org/abs/2404.15159>
- Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). *Layer Normalization*. <http://arxiv.org/abs/1607.06450>
- Lv, K., Yang, Y., Liu, T., Gao, Q., Guo, Q., & Qiu, X. (2023). *Full Parameter Fine-tuning for Large Language Models with Limited Resources*. <http://arxiv.org/abs/2306.09782>
- Ding, N., Qin, Y., Yang, G., Wei, F., Yang, Z., Su, Y., Hu, S., Chen, Y., Chan, C. M., Chen, W., Yi, J., Zhao, W., Wang, X., Liu, Z., Zheng, H. T., Chen, J., Liu, Y., Tang, J., Li, J., & Sun, M. (2023). Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature Machine Intelligence*, 5(3), 220–235. <https://doi.org/10.1038/s42256-023-00626-4>
- Hu, E., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (n.d.). *LoRA: Low-Rank Adaptation of Large Language Models*. <https://github.com/microsoft/LoRA>.
- Wei, J., Bosma, M., Zhao, V. Y., Guu, K., Yu, A. W., Lester, B., Du, N., Dai, A. M., & Le, Q. v. (2021). *Finetuned Language Models Are Zero-Shot Learners*. <http://arxiv.org/abs/2109.01652>
- Penedo, G., Malartic, Q., Hesslow, D., Cojocaru, R., Cappelli, A., Alobeidli, H., Pannier, B., Almazrouei, E., & Launay, J. (2023). *The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only*. <http://arxiv.org/abs/2306.01116>
- Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2022). *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers*. <http://arxiv.org/abs/2210.17323>
- Min, S., Lewis, M., Zettlemoyer, L., & Hajishirzi, H. (2021). *MetaICL: Learning to Learn In Context*. <http://arxiv.org/abs/2110.15943>
- Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., & Han, S. (2022). *SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models*. <http://arxiv.org/abs/2211.10438>

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). *Language Models are Few-Shot Learners*. <http://arxiv.org/abs/2005.14165>

Lisa Li, X., & Liang, P. (n.d.). *Prefix-Tuning: Optimizing Continuous Prompts for Generation*.
Lester, B., Al-Rfou, R., & Constant, N. (2021). *The Power of Scale for Parameter-Efficient Prompt Tuning*. <http://arxiv.org/abs/2104.08691>

Devlin, J., Chang, M.-W., Lee, K., Google, K. T., & Language, A. I. (n.d.). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*.
<https://github.com/tensorflow/tensor2tensor>

Goodfellow, I. J., Mirza, M., Xiao, D., Courville, A., & Bengio, Y. (2013). *An Empirical Investigation of Catastrophic Forgetting in Gradient-Based Neural Networks*.
<http://arxiv.org/abs/1312.6211>

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., & Lample, G. (2023). *LLaMA: Open and Efficient Foundation Language Models*. <http://arxiv.org/abs/2302.13971>

Huang, C., Liu, Q., Lin, B. Y., Pang, T., Du, C., & Lin, M. (2023). *LoraHub: Efficient Cross-Task Generalization via Dynamic LoRA Composition*. <http://arxiv.org/abs/2307.13269>

Zhang, J., Chen, S., Liu, J., & He, J. (2023). *Composing Parameter-Efficient Modules with Arithmetic Operations*. <http://arxiv.org/abs/2306.14870>

Petroni, F., Rocktäschel, T., Lewis, P., Bakhtin, A., Wu, Y., Miller, A. H., & Riedel, S. (n.d.). *Language Models as Knowledge Bases?* Association for Computational Linguistics.
<https://github.com/pytorch/fairseq>

Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., & Liu, T. (2023). *A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions*. <http://arxiv.org/abs/2311.05232>

Ravi, S. S., Mielczarek, B., Kannappan, A., Kiela, D., & Qian, R. (2024). *Lynx: An Open Source Hallucination Evaluation Model*. <http://arxiv.org/abs/2407.08488>

Almazrouei, E., Alobeidli, H., Alshamsi, A., Cappelli, A., Cojocaru, R., Debbah, M., Goffinet, É., Hesslow, D., Launay, J., Malartic, Q., Mazzotta, D., Noune, B., Pannier, B., & Penedo, G. (2023). *The Falcon Series of Open Language Models*. <http://arxiv.org/abs/2311.16867>

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. <http://arxiv.org/abs/2201.11903>

Liu, X., Zheng, Y., Du, Z., Ding, M., Qian, Y., Yang, Z., & Tang, J. (2021). *GPT Understands, Too*. <http://arxiv.org/abs/2103.10385>

APPENDIX A: RESEARCH PROPOSAL

Fine-Tuning the LLMs using Parameter Efficient Fine-Tuning (PEFT) methods

Swapnil Anil Janorkar

Research Proposal for
Master of Science in Machine Learning & Artificial Intelligence

Liverpool John Moores University & upGrad

May 2024

Abstract

Since the launch of ChatGPT by OpenAI in November 2022, a new boom in the Artificial Intelligence has been started especially in the field of Large Language and Foundation Models. Almost all the Natural Language Processing (NLP) tasks like Sentiment Analysis, Topic Modelling, Machine Translation, Named Entity Recognition (NER), etc. can be accomplished by modern LLMs. These LLMs are trained with vast amounts of generic data available on the internet and may not perform well with real-life business use cases. To cater to such business use cases LLMs need to be fine-tuned with private or enterprise data. In real life, all the data might not be available at once for the LLM fine-tuning process. Data might be available periodically. This research tries to find out whether LLMs should be fine-tuned more frequently with smaller datasets or less frequency with larger datasets. This research will help AI Engineers and Data Scientists to determine the frequency of fine-tuning and the amount of data to be used. LLM performance, resources required, and time taken for fine-tuning will be considered for determining the overall performance with both approaches.

1. Background

Since the introduction of ChatGPT (Generative Pre-Trained Transformers) in October 2022 OpenAI has started the Artificial Intelligence boom in the Foundation and Large Language Model (LLM) field. All the big tech companies have already developed various general-purpose LLMs; like Llama2 and Llama3 from Meta (Touvron et al., 2023), Gemini from Google, Claude from Anthropic, Mistral 7b, Mixtral 8x7b and Mixtral 8x22b from Mistral AI, falcon from tiuuac (Almazrouei et al., 2023).

These LLMs are trained on a large volume of data available on the internet. These general-purpose foundation models are good for performing a wide range of Natural Language Processing tasks. E.g. Question-Answer, Text Generation, Summarization, Translation, Text Completion, etc.

Furthermore, these LLMs can be re-trained to perform some specific tasks more efficiently and accurately using the process called LLM fine-tuning. GitHub Co-pilot and Meta Code Llama are LLMs specifically fine-tuned for writing programming code or providing coding suggestions.

So, in short, a general-purpose LLM can be turned into a specialist of a certain field using model fine-tuning. LLM fine-tuning is a data, resource, and time-intensive process. It requires an enormous volume of specialized datasets and large computing power. Mainly GPUs.

LLM has a wide variety of applications in the field of Natural Language Processing. Almost all the NLP tasks can be done using LLMs, but in some specific business cases LLMs need to fine-tune to achieve the desired results. e.g. Sentiment analysis, Named Entity Recognition, and Topic Modelling for custom datasets.

GitHub Co-Pilot which is based on OpenAI's GPT-3 and GPT-4, is fine-tuned using programming languages to generate the coding suggestions.

The purpose of this research is to find out the effect of the frequency of fine-tuning and the volume of datasets used on the LLM performance, resources required for fine-tuning, and total time taken. This study also aims to help Data Scientists and AI Engineers to decide the

frequency of fine-tuning and amount of data to be used for optimal use of resources without compromising the accuracy and performance of LLM.

2. Related Research OR Related Work

A comprehensive survey has been conducted to study the computational and implementation cost of various PEFT methods for fine-tuning the LLMs. This survey can help developers apply a cost-effective PEFT method for fine-tuning the LLMs. (Ding et al., 2023)

A detailed study has been done to compare the Few-shot in-context learning (ICL) and PEFT and it has been demonstrated that PEFT offers better accuracy as well as significantly lowers computational costs. (Liu et al., 2022)

LLM fine-tuning is resource-intensive task and needs expensive GPUs. Organizations leverage the public cloud service providers to rent the GPUs for fine-tuning purposes. This necessitates the sharing of sensitive enterprise data over the internet to the public cloud providers. DLoRA helps to perform scalable PEFT operations collaboratively between the user devices and cloud infrastructure. Coupled with the proposed Kill and Revive algorithm, the evaluation results demonstrate that DLoRA can significantly reduce the computation and communication workload over user devices while achieving superior accuracy and privacy protection. (Gao and Zhang, 2024)

PEFT methods only update a small number of weights in the LLM while keeping the rest of the model weights untouched. However, editing the representation of rich semantic information might be a more powerful alternative. Representation Fine-Tuning (ReFT) methods operate on a frozen base model and learn task-specific interventions on hidden representations. Low-Rank Linear Subspace ReFT (LoReFT) is a drop-in replacement for existing PEFTs and learns interventions that are 10x-50x more parameter-efficient than prior state-of-the-art PEFTs. (Wu et al., 2024)

As stated earlier many times LLM fine-tuning is a resource-intensive and costly affair. QLoRA stands for Quantized Low Rank Adaptation, it is an efficient fine-tuning approach. It can help to Fine-tune the 65B param model on a single 48GB GPU by reducing the memory usage on the other hand preserving full 16-bit Fine-tuning task performance. QLoRA backpropagates

gradients through a frozen, 4-bit quantized pre-trained language model into Low-Rank Adapters (LoRA) (Dettmers et al., 2023)

3. Aim and Objectives

In real life case large volume of training datasets might not be readily available for fine-tuning the LLM for some specific business use case. This research will help to decide the frequency of fine-tuning the LLM with domain-specific datasets and the volume of datasets to be used.

This research paper tries to find out model performance, model accuracy, and resource consumption when LLM is fine-tuned more frequently with smaller datasets versus less frequent fine-tuning with large datasets at once.

Objectives

1. To explore the Parameter-Efficient Fine-Tuning (PEFT) methods LoRA and QLoRA.
2. To fine-tune the model using the PEFT method using all the training datasets at once versus splitting the large dataset into multiple smaller datasets of equal size. Analyse both the approaches against following metric.
 1. CPU/GPU requirement.
 2. Memory requirement.
 3. Time taken to complete the Fine-tuning.
 4. Model accuracy and performance.

4. Significance of the Study

Since the emergence of Large Language Models, the field of natural language processing has transformed because of the advanced and highly sophisticated capabilities of LLMs. These models are trained on large numbers of datasets and able to perform various range of tasks including but not limited to text translation, generation, summarization, and Q&A. Although LLMs perform well at doing generic tasks but they fail to perform in specific tasks or domains. Fine-tuning helps to turn the pre-trained LLM into performing more specialized tasks. By fine-tuning the LLM on specialized datasets, performance in that domain is greatly improved while preserving the original knowledge of the LLM.

In practice all the specialized data required for fine-tuning might not be present at once, it might be available at regular intervals in batches. So, deciding the frequency of LLM fine-tuning and the amount of data to be used is crucial as fine-tuning is a resource and time intensive and costly process.

This research tries to find out the effect of frequent fine-tuning on smaller datasets versus infrequent fine-tuning using larger datasets on LLM performance and resource requirements. This research will guide engineers in deciding the frequency and amount of data to be used for optimal model performance and resource utilization.

5. Scope of the Study

The scope of this research work is defined as follows.

- Only open source LLM models will be used for fine-tuning. Like Meta's Llama 2, Llama 3, Mixtral from Mistral AI, and Falcon from tiuuue.
- This experiment will consider Performance-Efficient Fine-Tuning methods like LoRA and QLoRA to implement fine-tuning.
- Open-source software and libraries will be used for implementation.
- Publicly available GPUs such as Google Collab and Kaggle will be used for fine-tuning.
- The project will be completed within 17 weeks after the research proposal submission.

Out of scope

- Understanding of model hallucination because of fine-tuning
- Implementation of PEFT using another method than LoRA and QLoRA, like prefix tuning, P-tuning, prompt tuning, IA3

6. Research Methodology

Main objective of this research is to compare LLM performance when LLM is fine-tuned more frequently with smaller datasets versus less frequency with large volume of datasets.

6.1 Datasets

The following datasets will be used for Fine-tuning the LLMs.

6.1.1 microsoft/orca-math-word-problems-200k

This dataset was created by the Microsoft Research team in Feb 2024. This dataset has around 200,000 school-level math word questions and answers. This dataset has been tailored to improve the mathematical capabilities of the LLMs. Every data record in the dataset has a math question and respective answer.

Data Fields:

The dataset has the following fields.

- Question: Mathematical question in word form.
- Answer: Answer to the mathematical question.

6.1.2 Vezora/Tested-22k-Python-Alpaca

This dataset consists of 22,000 working Python code samples. This dataset helps to build robust Python code LLM something like GitHub Co-Pilot but only concentrating on Python programming language.

Data Fields:

The dataset has the following fields.

- Instruction: Problem statement to write Python code.
- Input: An input to validate the Python code.
- Output: Python code for the given problem statement.

6.2 Algorithms and Technical Description

6.2.1 Large Language Models (LLM)

The LLM is a general-purpose foundation model pre-trained on large amounts of data, that can be used to understand, summarize, generate, and predict the text contents. LLMs are also capable of performing other Natural Language Processing (NLP) tasks like classification, translation, topic modeling, and text completion.

LLMs are artificial neural networks built with decoder-only transformer-based architecture. Transformers are the basic building block of all the language models with Transformers (LLMs). Transformer architecture was introduced by Google in December 2017 in the Attention Is All You Need paper.

LLMs are scalable to be fine-tuned to perform specific task to improve their performance in some specific field/domain. A specialized domain-specific data is used for fine-tuning so that the model can answer the questions or perform tasks about that specific field/domain. e.g. finance, medical, legal, sports, technology, etc.

GPT 3.5, GPT 4, Llama 2, Llama 3, Mistral8b, Gemini are some of the popular LLMs currently available in the market.

6.2.2 Fine-Tuning

In LLM fine-tuning already pre-trained LLM is re-trained with additional domain-specific data, so that model can learn new capabilities. Some or all the model weights are updated in the fine-tuning process based on the approach used. Fine-tuning helps to refine the LLM capabilities and improves the performance in a particular task or domain. In short, fine-tuning is a process of turning a general-purpose model into a specialized model.

Fine-tuning promotes transfer learning which is why it requires fewer computational resources, time, and money compared to training the LLM model from scratch. Even smaller organizations can Fine-tune the LLM for their domain-specific use cases.

A lower learning rate is used in Fine-tuning than the initial pre-training. This lower learning rate helps to minimize the overall loss function.

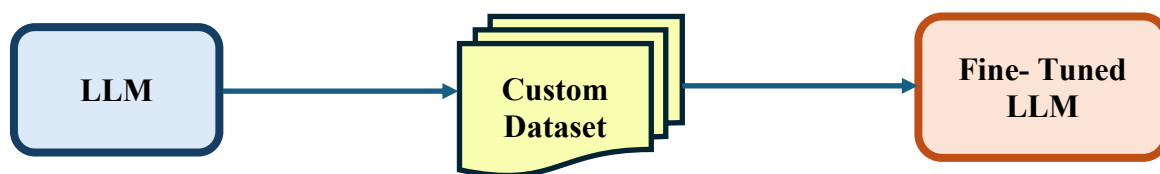


Figure 1: LLM Fine-tuning

6.2.3 Parameter-Efficient Fine-tuning (PEFT)

Fine-tuning helps to achieve transfer learning; parameters of pre-trained models are further trained on new data to understand new trends and patterns. Training an LLM is a computationally extensive task. Memory is required not only to store the model but also the parameters that are necessary for the training process. For simple hardware, it's not possible to handle such memory and computationally demanding tasks.

This is why PEFT is important. Full Parameter Fine-Tuning updates all the model parameters on the other hand PEFT method only updates a smaller subset of model parameters keeping most of the other parameters untouched. Typically, only 10-15% of the model parameters are updated by PEFT in fine-tuning, keeping the rest of the parameters frozen.

Because of PEFT, the computation and storage required for the model fine-tuning process reduces drastically thus lowering the cost of fine-tuning significantly. PEFT not only reduces the computational and storage required but also deals with catastrophic forgetting. As it does not change the original LLM, the model does not forget the previously learned knowledge.

6.2.3.1 Low Ranking Adaptation (LoRA)

LoRA introduces a new additional layer of trainable parameters for model training while keeping the original model weights frozen. These additional parameters are of reduced rank (Dimensions) compared to the dimensions of the original network. This reduces the number of trainable parameters significantly, thus reducing the computation and storage requirements.

Consider there are 256×256 trainable parameters in the feedforward network, making the total number of trainable parameters equal to 65,536. These original parameters are kept frozen during fine-tuning and the LoRA adapter with rank 2 is added. So now new training parameters

are 512×2 for W_A and 512×2 for W_B . Thus, the total number of trainable parameters are $W_A \times 1024 + W_B \times 1024 = 2048$. These new 2048 parameters are fine-tuned with domain-specific data.

Finally, once the fine-tuning is completed, these new parameters are multiplied with the original frozen layer.

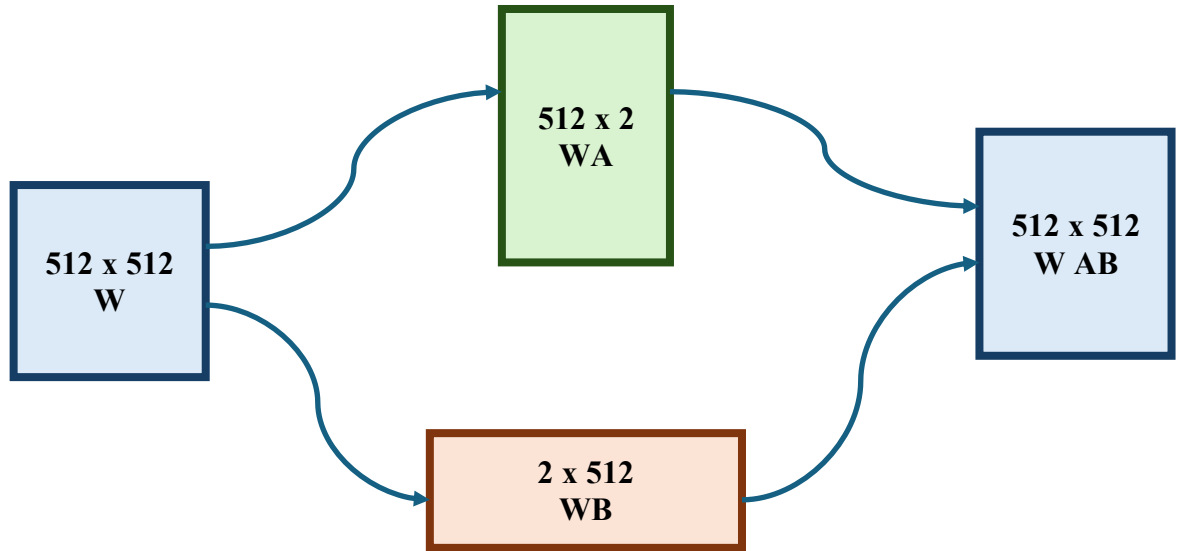


Figure 2: Parameter Efficient Fine-Tuning Using LoRA

6.2.3.2 Quantized Low-Rank Adaptation (QLoRA)

QLoRA can be called an advanced version of LoRA, which requires even less computing and storage than LORA. QLoRA takes a LoRA one step ahead by also quantizing the weights of LoRA adapters (smaller matrices) to lower precision. E.g. 4-bit instead of 8-bit used by LoRA. This further reduces the memory required and storage needed almost by half. In QLoRA the pre-trained model (LLM) is loaded into GPU memory with quantized 4-bit weights, in contrast to 8 bits used in LoRA. Even though the bit precision is reduced in QLoRA, it does not affect the effectiveness or accuracy of the model significantly. The effectiveness and accuracy of QLoRA are comparable with the effectiveness and accuracy of LoRA fine-tuning.

With QLoRA it is possible to fine-tune a large 65B parameter model on a single GPU with just 48GB memory, without any loss in quality compared to full 16-bit training. Additionally, QLoRA makes it feasible to fine-tune large models with full 16-bit precision on standard

academic setups, paving the way for more exploration and practical uses of large language models (LLMs).

6.2.4 Implementation:

One or more open-source Large Language Models (LLMs) will be fine-tuned with the data mentioned in Section 4.4.

e.g. Llama 2, Llama 3 from Meta, Mistral 7B, Mixtral 8x7B from MistralAI, falcon 7B from Technology Innovation Institute UAE. (tiiuae)’

Parameter-Efficient Fine-Tuning methods for LLM mentioned in section 4.2.3 will be used for fine-tuning. LLMs will be fine-tuned using the following two approaches.

6.2.4.1 Using a complete dataset

All the data in the dataset will be used to fine-tune the LLM.

6.2.4.2 Splitting the dataset into multiple smaller datasets

The dataset will be split into multiple smaller datasets of equal size and these smaller sub datasets will be used for fine-tuning.

Each smaller dataset will be treated as an independent dataset. After each iteration of the fine-tuning process, a model will be saved and for every iteration, a previously fine-tuned model will be considered as a new base model.

6.3 Evaluation

The LLM fine-tuning approaches mentioned in Section 4.2.4 will be evaluated using the following metric.

- CPU/GPU required.
- Memory required.
- Total time taken to complete fine-tuning.
- Accuracy & performance.

7. Resource Requirement

7.1 Hardware

The following hardware resources will be required for the research.

- Laptop or desktop with internet access and software for writing/editing the documents.
- GPU/CPU for Fine-tuning the LLMs.

Note: Publicly available GPUs in Kaggle (T4 x2 and P100) and Google Collab will be used for LLM fine-tuning.

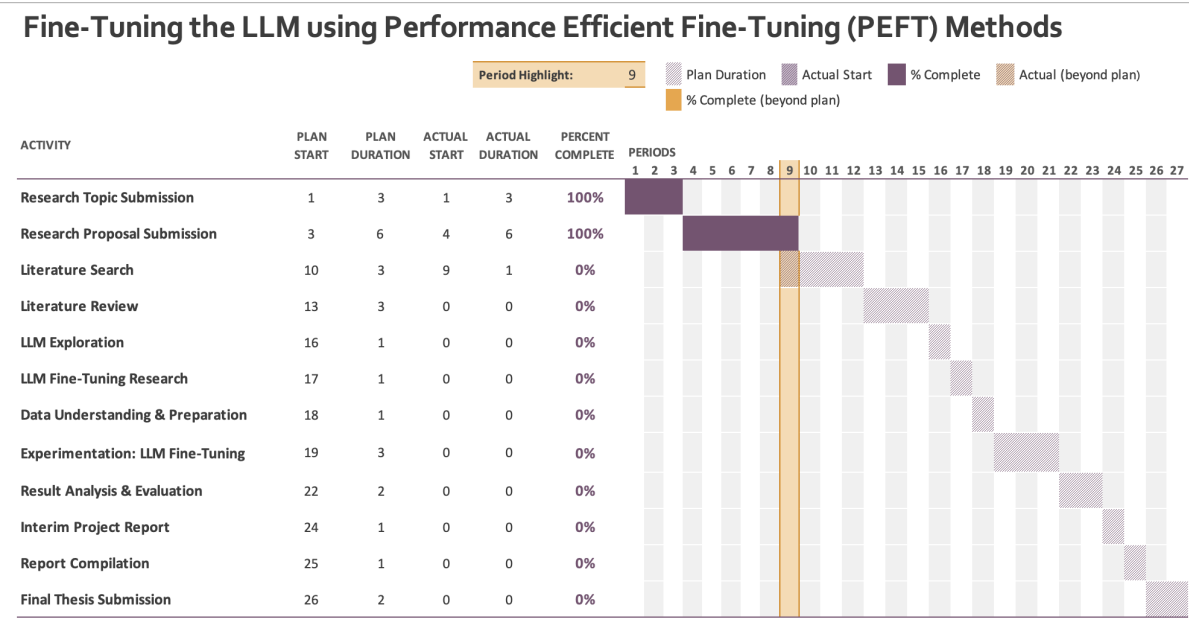
7.2 Software

The following software resources will be required for the research.

- Web Browser
- Jupyter Notebook
- Python 3.0+
- NVIDIA CUDA
- Deep learning libraries. e.g. tensorflow, pytorch, huggingface
- Python libraries for data analysis and visualization. E.g numpy, pandas, matplotlib, seaborn.
- Python libraries for LLM finetuning. E.g Bitsandbytes, transformers, peft, accelerate, datasets, einops
- Open Source LLM Models. E.g. Meta's Llama2/Llama3, Mistral AI's Mistral 7b, Mixtral 8x7b

8. Research Plan

8.1 Project Planning and Estimation



9. References

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention Is All You Need*. <http://arxiv.org/abs/1706.03762>
- Han, Z., Gao, C., Liu, J., Zhang, J., & Zhang, S. Q. (2024). *Parameter-Efficient Fine-tuning for Large Models: A Comprehensive Survey*. <http://arxiv.org/abs/2403.14608>
- Gao, C., & Zhang, S. Q. (2024). *DLoRA: Distributed Parameter-Efficient Fine-tuning Solution for Large Language Model*. <http://arxiv.org/abs/2404.05182>
- Liu, H., Tam, D., Muqeeth, M., Mohta, J., Huang, T., Bansal, M., & Raffel, C. (2022). *Few-Shot Parameter-Efficient Fine-tuning is Better and Cheaper than In-Context Learning*. https://proceedings.neurips.cc/paper_files/paper/2022/hash/0cde695b83bd186c1fd456302888454c-Abstract-Conference.html
- Wu, Z., Arora, A., Wang, Z., Geiger, A., Jurafsky, D., Manning, C. D., & Potts, C. (2024). *ReFT: Representation Finetuning for Language Models*. <http://arxiv.org/abs/2404.03592>
- Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs*. https://proceedings.neurips.cc/paper_files/paper/2023/hash/1feb87871436031bdc0f2beaa62a049b-Abstract-Conference.html
- Aghajanyan, A., Zettlemoyer, L., & Gupta, S. (2020). *Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-tuning*. <http://arxiv.org/abs/2012.13255>
- Grosjean, J., & Vamvas, J. (2024). *Fine-tuning the SwissBERT Encoder Model for Embedding Sentences and Documents*. <http://arxiv.org/abs/2405.07513>
- Xu, J., Saravanan, K., van Dalen, R., Mehmood, H., Tuckey, D., & Ozay, M. (2024). *DP-DyLoRA: Fine-tuning Transformer-Based Models On-Device under Differentially Private Federated Learning using Dynamic Low-Rank Adaptation*. <http://arxiv.org/abs/2405.06368>

- Jie, S., Tang, Y., Ding, N., Deng, Z.-H., Han, K., & Wang, Y. (2024). *Memory-Space Visual Prompting for Efficient Vision-Language Fine-tuning*. <http://arxiv.org/abs/2405.05615>
- Gekhman, Z., Yona, G., Aharoni, R., Eyal, M., Feder, A., Reichart, R., & Herzig, J. (2024). *Does Fine-tuning LLMs on New Knowledge Encourage Hallucinations?* <http://arxiv.org/abs/2405.05904>
- Liu, Z., & Luo, J. (2024). *AdaMoLE: Fine-tuning Large Language Models with Adaptive Mixture of Low-Rank Adaptation Experts*. <http://arxiv.org/abs/2405.00361>
- Yang, Y., Liu, X., Gao, T., Xu, X., & Wang, G. (2024). *SA-FedLora: Adaptive Parameter Allocation for Efficient Federated Learning with LoRA Tuning*. <http://arxiv.org/abs/2405.09394>
- Tian, C., Shi, Z., Guo, Z., Li, L., & Xu, C. (2024). *HydraLoRA: An Asymmetric LoRA Architecture for Efficient Fine-tuning*. <http://arxiv.org/abs/2404.19245>
- Wu, X., Huang, S., & Wei, F. (2024). *Mixture of LoRA Experts*. <http://arxiv.org/abs/2404.13628>
- Gao, Z., Wang, Q., Chen, A., Liu, Z., Wu, B., Chen, L., & Li, J. (2024). *Parameter-Efficient Fine-tuning with Discrete Fourier Transform*. <http://arxiv.org/abs/2405.03003>
- Zhao, J., Wang, T., Abid, W., Angus, G., Garg, A., Kinnison, J., Sherstinsky, A., Molino, P., Addair, T., & Rishi, D. (2024). *LoRA Land: 310 Fine-tuned LLMs that Rival GPT-4, A Technical Report*. <http://arxiv.org/abs/2405.00732>
- Li, D., Ma, Y., Wang, N., Cheng, Z., Duan, L., Zuo, J., Yang, C., & Tang, M. (2024). *MixLoRA: Enhancing Large Language Models Fine-tuning with LoRA based Mixture of Experts*. <http://arxiv.org/abs/2404.15159>

APPENDIX B: LLM FINE-TUNING IMPLEMENTATION IN PYTHON

GitHub Link to the Source Code:

<https://github.com/sjanorkar/upgrad-ljmu-final-thesis>